केन्द्रीय विनिर्माणकारी प्रौद्योगिकी संस्थान
CENTRAL MANUFACTURING TECHNOLOGY INSTITUTE
(An Autonomous R&D Institute
Under Ministry of Heavy Industries, Govt. of India)
SINCE 1962

CENTRAL MANUFACTURING TECHNOLOGY INSTITUTE

Tumkur Road, Bengaluru – 560022

PROJECT REPORT

Submitted by

GAGAN SURYA KUMAAR S

Email: sgagansuryakumaar@gmail.com

ST. JOSEPH ENGINEERING COLLEGE

Under the Guidance of

Mr. Tom Thampy

Scientist-E

(C-ASMP, CMTI)

DURING THE PERIOD

12-02-2024 To 12-08-2024

BONAFIDE CERTIFICATE

This is certified that Mr. Gagan Surya Kumaar S has worked on the projects titled “Autonomous agricultural machinery for enhancing Plant Leaf Disease Detection in real time, Advancing Drone Technology through automation” and “Mass flow controller” under my direct supervision. I at this moment affirm that the work presented in this study is the genuine and actual effort of Gagan Surya Kumaar S.

Mr. Tom Thampy
SCIENTIST E (C-ASMP)

Date:12/08/2024
Place: Bengaluru.

PROJECT WORK COMPLETION LETTER

Date: 12/08/2024

Subject: Request for the Project internship completion certificate

Respected Sir,

My Name is Gagan Surya Kumaar S, completed my BE in Electronics and Communication Engineering at St. Joseph Engineering College, Vamanjoor, Mangalore. I'm thankful to CMTI for providing me the opportunity to carry out my Project on the topic “Drones and Mass flow Controllers” under the guidance of **Mr. Tom Thampy** Scientist-E at the Department of Additive and Special Manufacturing Process (ASMP).

I am grateful for all the experience and knowledge I have received during my internship work from 12-02-2024 To 12-08-2024.

I hereby request that you kindly issue the Project internship completion certificate.

Thanking you,

Yours Sincerely,

Gagan Surya Kumaar S,

Project student

ASMP Department

Mr. TOM THAMPY
SCIENTIST E (C-ASMP)

ACKNOWLEDGEMENT

I am deeply thankful to the director of the Central Manufacturing Technology Institute (CMTI), **Dr. Nagahanumaiah**, for allowing me to be a part of CMTI as a project intern. The project work experience at CMTI has been an exceptional chance for my learning and professional development, and I am truly grateful for the exposure it has provided me.

I express my sincere gratitude to my supervisor, **Mr. Tom Thampy**, Scientist-E group head at the additive and special manufacturing process department (C-ASMP) whose invaluable guidance, unwavering support, and boundless encouragement have been instrumental in shaping the outcome of this project. His perceptive motivation and everlasting patience have continuously inspired me to stay motivated and strive for excellence in designing this project.

I would like to express my deepest gratitude to my supervisor, **Mr. Emmanuel GospelRaj**, Scientist-B (C-ASMP), for his invaluable guidance, unwavering support, and boundless encouragement throughout this project. His insightful motivation and enduring patience have been a constant source of inspiration, driving me to stay focused and strive for excellence in every aspect of the project's design.

I am deeply thankful to **Dr. Dayakshini**, Professor and Head, Department of Electronics and Communication Engineering, St. Joseph Engineering College, Vamanjoor, Mangalore for providing me with the opportunity to be a part of CMTI as a project intern.

I am deeply thankful to **Mr. Glenson Toney**, Assistant Professor, Department of Electronics and Communication Engineering, St. Joseph Engineering College, Vamanjoor, Mangalore for providing me with the opportunity to be a part of CMTI as a project intern.

I am confident that the practical experience I have gained at CMTI will greatly contribute to my future success in the industry. I am excited to apply the valuable lessons learned from this internship to future projects and continue growing as a professional.

Gagan Surya Kumaar S

CMTI

Central Manufacturing Technology Institute, CMTI, is a Research & Development organization focusing on providing Technology Solutions to the manufacturing sector and assisting technological growth in the country. CMTI| plays a key role in applied research, design, and development (RD&D), technology forecasting, assimilation, and dissemination of manufacturing technology to Indian industries.

CMTI is a registered Government of India Society, an autonomous institution under the administrative control of the Ministry of Heavy Industries, Government of India, governed by a Governing Council which has representation from the machine tool manufacturing and user industries, the Union Government, and the Government of Karnataka. The Governing Council evolves Policies and monitors policy deployment. A Research Advisory Board (RAB), a Technical Committee with representatives| from industries and academia, assists the institute on matters relating to technology advancement.

Drishti is an online platform developed by CMTI that provides a synergy between industries and young innovators to solve complex challenges. CMTI, under the SAMARTH Udyog Bharat 4.0 Platform of the Department of Heavy Industry (DHI) is setting up a Smart Manufacturing Demo &Development Cell (SMDDC) as a Common Engineering Facility Centre (CEFC) to develop, propagate, and support the process of adoption of smart manufacturing practices, by the rapidly growing Indian manufacturing industry. Skilling and Re-Skilling in Smart Manufacturing is one of the focal activities of SMDDC.

Table of Contents

INTRODUCTION.....9

PROJECT 1 10

AUTONOMOUS AGRICULTURAL MACHINERY FOR ENHANCING PLANT LEAF DISEASE DETECTION IN REAL TIME

Problem Definition.....10

Objectives 11

Methodology 11

TRANSMITTER AND RECEIVER

System Design 12

Simulation.....13

Experimental Setup.....15

Prototyping.....16

Testing and Validation.....17

FLIGHT CONTROLLER USING ARDUINO UNO

Why diode and resistors? 18

Reading signals from the receiver by interfacing with Arduino Uno r3.....18

To use interrupts referring from the interrupt Vector table 21

Optimising the code for ESCs.....23

PID Control System 30

DATA COLLECTION

Snapshots of coconut and arecanut leaf dataset.....34

Classifications.....36

Modelling.....36

Implementation 37

VGG16 model training 38

Model evaluation 40

PROJECT 2 41
ADVANCING DRONE TECHNOLOGY THROUGH AUTOMATION

Objectives 41

FLIGHT CONTROLLER USING STM32F103C8T6 MICROCONTROLLER

Programming a STM32F103C8T6 microcontroller 42

Connecting FTDI programmer to STM32 board 42

Uploading the program to STM32 board 43

Telemetry connection and coding 44

The gyro - Auto levelling 47

PROJECT 3 54
MASS FLOW CONTROLLER

Objectives 55

Circuit diagram 55

Code explanation 56

CONCLUSIONS 60

APPENDIX 61

ABSTRACT

This paper presents the design and implementation of an innovative plant leaf disease detection robot utilizing a drone for transportation and a Raspberry Pi paired with a camera module for image processing. The system aims to enhance agricultural productivity by providing early detection of plant diseases, thus enabling timely intervention. The drone is equipped with a Raspberry Pi, which processes images captured by the camera module using advanced image processing algorithms. These algorithms analyze the captured images to identify symptoms of various plant diseases. The integration of drone technology ensures wide coverage and accessibility to hard-to-reach areas, while the Raspberry Pi's computational power facilitates real-time analysis. Experimental results demonstrate the effectiveness of the system in accurately detecting common plant diseases, highlighting its potential to revolutionize modern agricultural practices by offering a scalable and efficient solution for crop health monitoring.

Moreover, design of an autonomous drone system utilizing the STM32F103C8T6 microcontroller to develop a robust and efficient platform for various applications such as aerial surveillance, environmental monitoring, and agricultural assessment. The STM32 microcontroller serves as the core of the flight control system, providing the computational power needed for real-time processing of sensor data and execution of complex flight algorithms. The drone is equipped with a suite of sensors, including GPS, IMU (Inertial Measurement Unit), and ultrasonic sensors, to facilitate autonomous navigation and obstacle avoidance. The integration of these components allows the drone to perform precise maneuvers and maintain stability in diverse operating conditions. The proposed system is evaluated through a series of test flights, demonstrating its capability to perform autonomous missions with high accuracy and reliability. This work highlights the potential of STM32-based drones in enhancing the efficiency and effectiveness of various autonomous aerial applications.

Additionally, development of a mass flow controller employing the AFM3020 sensor and a proportional valve for precise fluid flow regulation. The system is designed to provide accurate control of mass flow rates in various industrial and laboratory applications. The AFM3020 sensor offers high-resolution measurements of the flow rate, which are used to dynamically adjust the proportional valve, ensuring precise flow control. The controller integrates real-time feedback from the sensor to modulate the valve position, achieving desired flow rates with high accuracy and stability. This approach combines advanced sensing technology with precise valve control, offering a reliable and efficient solution for applications requiring meticulous flow management.

INTRODUCTION

The plant leaf disease detection robot represents a cutting-edge advancement in agricultural technology by combining drone capabilities with sophisticated image processing. This system utilizes a drone equipped with a high-resolution camera module to capture detailed images of plant leaves from multiple angles. The images are then analyzed in real-time by a Raspberry Pi, which uses advanced algorithms to accurately diagnose plant diseases. This integration of aerial mobility with powerful computing allows for efficient, precise disease detection, enabling farmers to proactively address plant health issues and manage crops effectively across extensive areas.

In parallel, the development of an autonomous drone using the STM32F103C8T6 microcontroller marks a significant leap in UAV technology. Renowned for its high processing power, low energy consumption, and real-time performance, the STM32F103C8T6 microcontroller manages complex tasks such as flight control, navigation, and obstacle avoidance. The drone's autonomous capabilities, bolstered by advanced sensors, GPS modules, and communication systems, make it suitable for a range of missions including surveillance, mapping, and delivery. This project showcases the STM32F103C8T6's versatility and the potential of autonomous drones to transform various sectors through intelligent, scalable aerial solutions.

Furthermore, mass flow controllers (MFCs) equipped with the AFM3020 sensor and a proportional valve offer an innovative approach to precise gas flow regulation across industrial applications. The AFM3020 sensor provides exceptional accuracy in measuring gas flow rates, while the proportional valve allows for real-time adjustments to maintain optimal flow control. This combination ensures high efficiency and stability in processes requiring stringent flow management, such as in semiconductor manufacturing, chemical processing, and laboratory research, enhancing overall system performance and reliability.

PROJECTS

1) Autonomous agricultural machinery for enhancing Plant Leaf Disease Detection in real time:

PROBLEM DEFINITION

Plant diseases continue to be a significant challenge for agricultural productivity worldwide. In India alone, an estimated 20-30% of crop yield is lost due to various plant diseases. According to the Food and Agriculture Organization (FAO), these diseases affect approximately 14.1% of global crops, leading to substantial economic consequences. The annual agricultural trade losses associated with plant diseases amount to a staggering $220 billion. Detecting and diagnosing plant diseases is crucial for effective management, but traditional methods only achieve an accuracy rate of 73% when identifying diseases based on plant leaf appearance. Given the scale of the problem and its impact on food security and economic stability, there is a growing need for innovative and reliable solutions to improve disease detection and control in agriculture.

Solution:

- UAV (Unmanned Aerial Vehicle) to detect and identify the type of disease in plants.
- Improving the accuracy of plant leaf disease detection using Image processing.

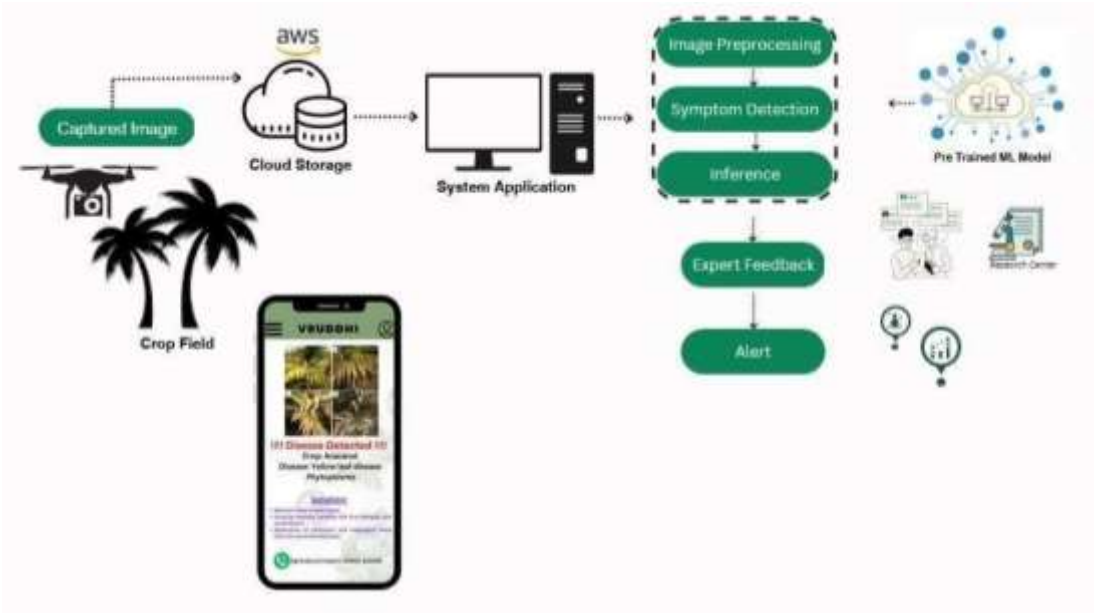
OBJECTIVE

The objectives of the project are:

1. Develop a cost-effective agricultural drone system for capturing high-resolution images of agricultural fields.
2. Implement image processing and analysis algorithms to detect underlying diseases, pests, or abnormalities in the captured field images.
3. Establish a secure and user-friendly communication platform through the Vruddhi mobile application, allowing agricultural scientists to authenticate disease detection results and providing farmers with real-time disease status updates and recommended measures.

METHODOLOGY

The advent of autonomous agricultural machinery has revolutionized the farming industry by providing efficient and precise solutions to various farming challenges. One crucial aspect of modern agriculture is the timely detection and analysis of plant diseases, as they can have devastating effects on crop yields and quality. To address this issue, the objective of this study is to develop an autonomous agricultural machinery system that enhances plant disease detection and enables real-time analysis. This system aims to improve the overall efficiency and productivity of farming practices while minimizing the negative impacts of plant diseases. The methodologies employed in this research will involve the integration of advanced technologies such as computer vision, machine learning, and real-time data analysis to create a comprehensive solution for plant disease management in agriculture. By combining cutting- edge technologies with autonomous machinery, farmers can obtain early disease detection, accurate identification, and timely intervention strategies, ultimately leading to improved crop health and increased yields.

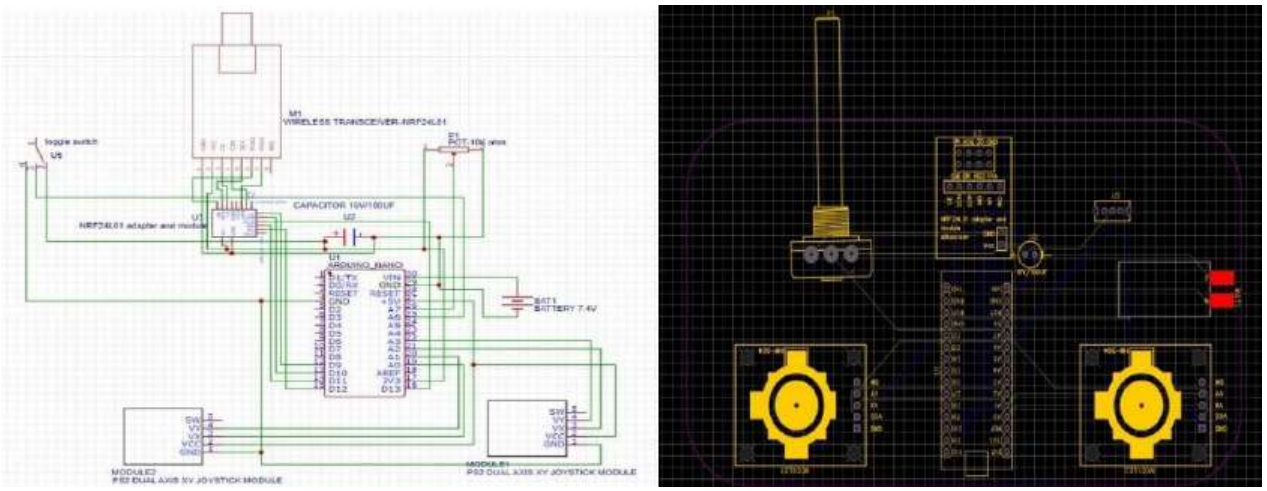


Methodology

System Design:

Transmitter (Tx):

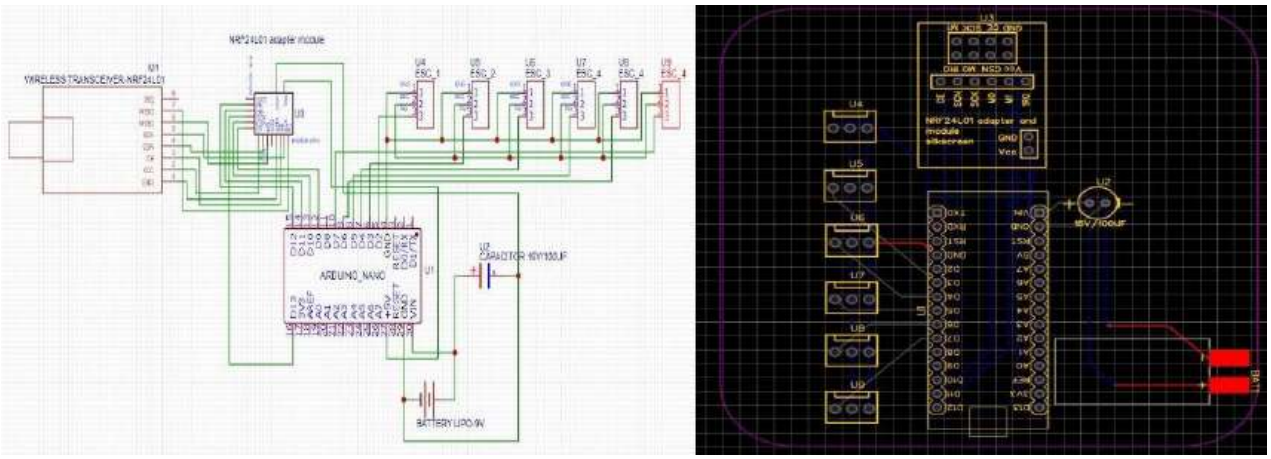
The transmitter, a handheld device, serves as the primary interface between the operator and the quadcopter. Featuring control sticks, switches, and an antenna, it operates within specific radio frequencies, typically around 2.4GHz. With configurable modes like mode 1 and mode 2, the transmitter offers pilots precise control over throttle, pitch, roll, and yaw, crucial for maneuvering the quadcopter effectively. Its ergonomic design and intuitive controls empower operators to navigate the quadcopter with accuracy and responsiveness, enhancing the overall flight experience.



Transmitter Circuit and PCB Design

Receiver (Rx):

Onboard the quadcopter, the receiver plays a pivotal role in receiving and interpreting control signals transmitted by the transmitter. Equipped with an antenna, channels, and signal processing circuitry, the receiver ensures seamless communication with the transmitter. By matching specific transmitters for compatibility and interfacing with the flight controller, the receiver translates received signals into actionable commands, dictating the quadcopter's movements. This real-time interaction between the transmitter and receiver enables remote piloting with precision, facilitating smooth and controlled flights while ensuring safety and reliability.



Receiver Circuit and PCB Design

Simulation

For the simulation of our transmitter and receiver system we used Arduino IDE for programming and simulation tools like Tinker cad.

Simulation Tools:

• Arduino IDE: Used for programming the Arduino Nano microcontrollers.

Simulation Parameters and Inputs:

1. Components:

• Arduino Nano (for both transmitter and receiver)

• RF transmitter and RF receiver module i.e., NRF24L01 + PA

• 3-way Toggle switch

• Serial Monitor (To check the Output)

2. Simulation Inputs:

• Analog signals from ps2 joysticks simulating user inputs.

• Simulated RF signals transmitted from the transmitter module.

• Reception of RF signals by the receiver module.

Simulation Methodology and Steps:

1. Circuit Design:

• Assemble the circuits for both the transmitter and receiver in breadboard.

• Place Arduino Nano on the breadboard along with the RF modules, joysticks and necessary components for both Tx and Rx.

• Connect the components according to the circuit diagram.

2. Simulation Setup and Programming:

• Write the Arduino code for both the transmitter and receiver functionalities in the Arduino IDE.

• Program the transmitter to read inputs from joysticks, encode data, and transmit it using the RF module.

• Program the receiver to receive the transmitted data, decode it, and control the output devices accordingly.

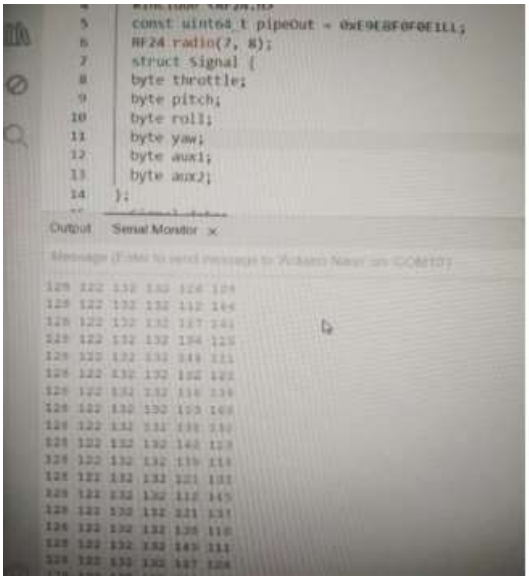
• Upload the Arduino code to the respective Arduino Nano boards in the simulation environment.

3. Simulation Execution:

• Start the simulation and observe the behaviour of the transmitter and receiver.

• Simulate joysticks motion on the transmitter side and observe the corresponding rotation of the motor on the receiver side as well as change in values on the serial monitor.

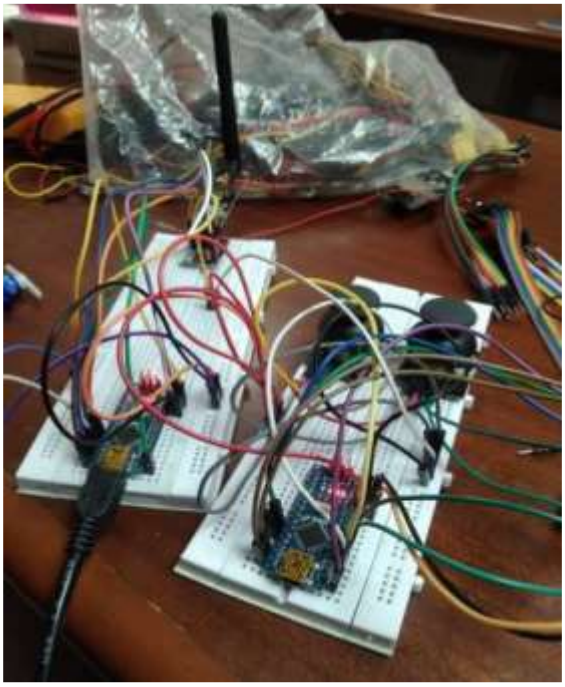
• Ensure that the communication between the transmitter and receiver is successful, and the transmitted data is accurately received and processed.



Transmitter and Receiver Simulation

Experimental Setup

Transmitter and Receiver Setup:



Transmitter and Receiver Experimental Setup

Protocol used in the nRF24L01 module for our transmitter and receiver is simple point-to-point communication protocol:

A simple point-to-point communication setup using the nRF24L01 modules, data is transmitted wirelessly from one module (transmitter) to another (receiver).

STEPS:

1. Initialization:

• Both the transmitter and receiver nRF24L01 modules are configured with compatible settings, such as the operating channel, data rate, payload size, and addresses. These settings must match on both ends for successful communication.

2. Transmitter Side:

• The transmitter prepares the data to be sent. This is the readings from the joysticks.

• The data is formatted into a packet suitable for transmission. This packet typically includes the payload data along with any necessary header information.

• The transmitter instructs the nRF24L01 module to transmit the packet over the air using its built-in radio transmitter.

3. Receiver Side:

• The receiver continuously listens for incoming data packets using the nRF24L01 module's built-in radio receiver.

• Upon receiving a packet, the receiver extracts the payload data and processes it as needed. This involve displaying Throttle, yaw, pitch and roll values which are adjusted at the transmitter side.

4. Repeat:

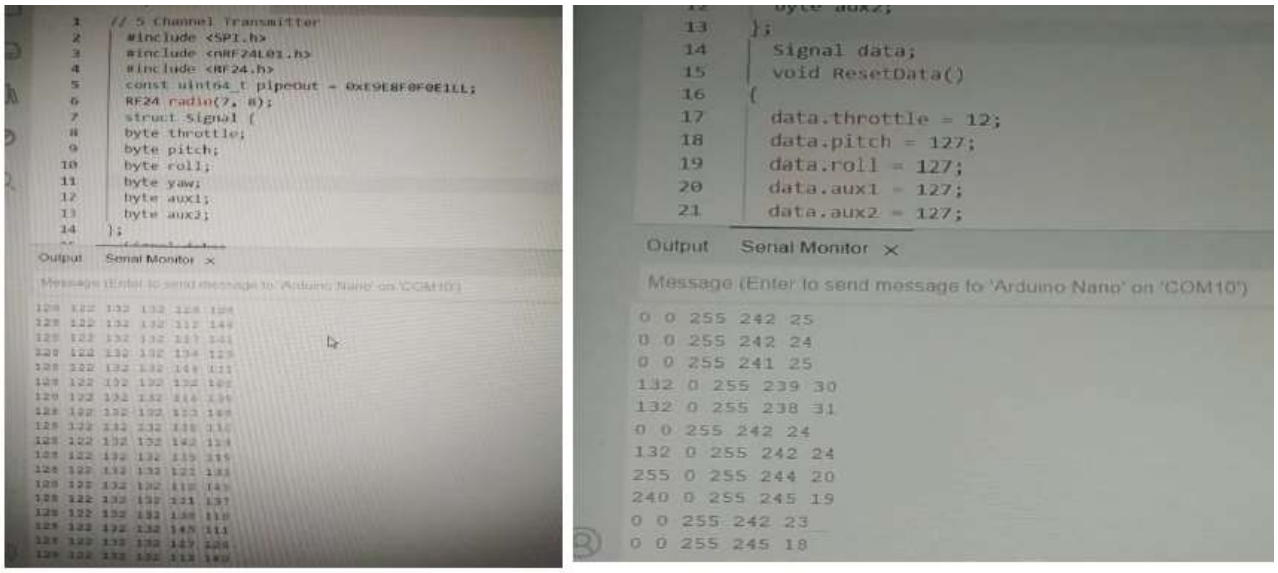
• The transmitter can continue to send data packets periodically, and the receiver continues to listen for incoming packets, allowing for continuous communication between the two modules.

Prototyping

a) Transmitter and Receiver hardware connection, b) Code compilation
c) & d) Vero Board integration of Tx and Rx

Testing and validation:

Transmitter Testing: -

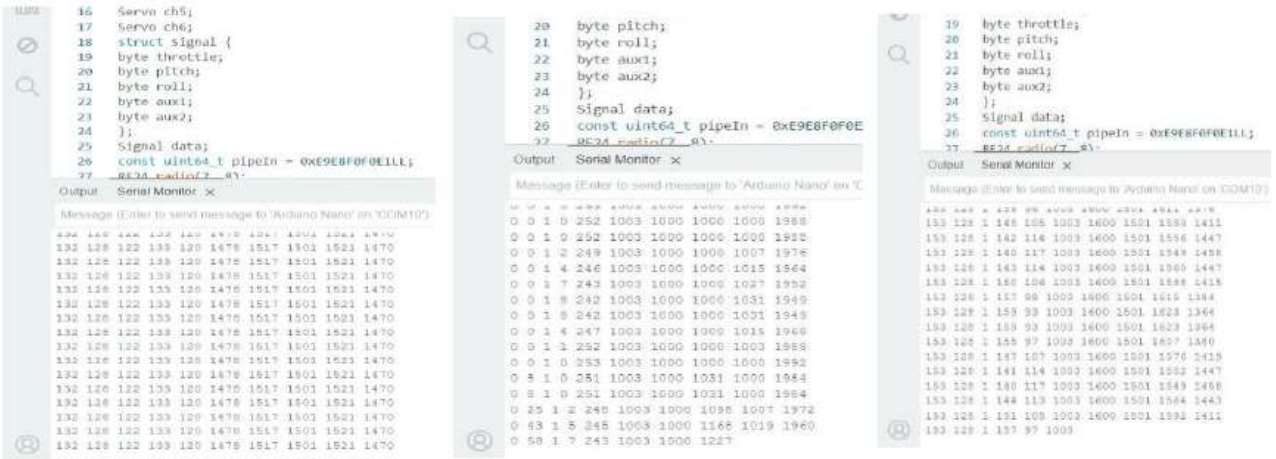


a) b)

a) Initial Values of Tx before testing, b) Tx responding to joystick inputs (i.e., Throttle, yaw, pitch and roll position)

Receiver Testing: -

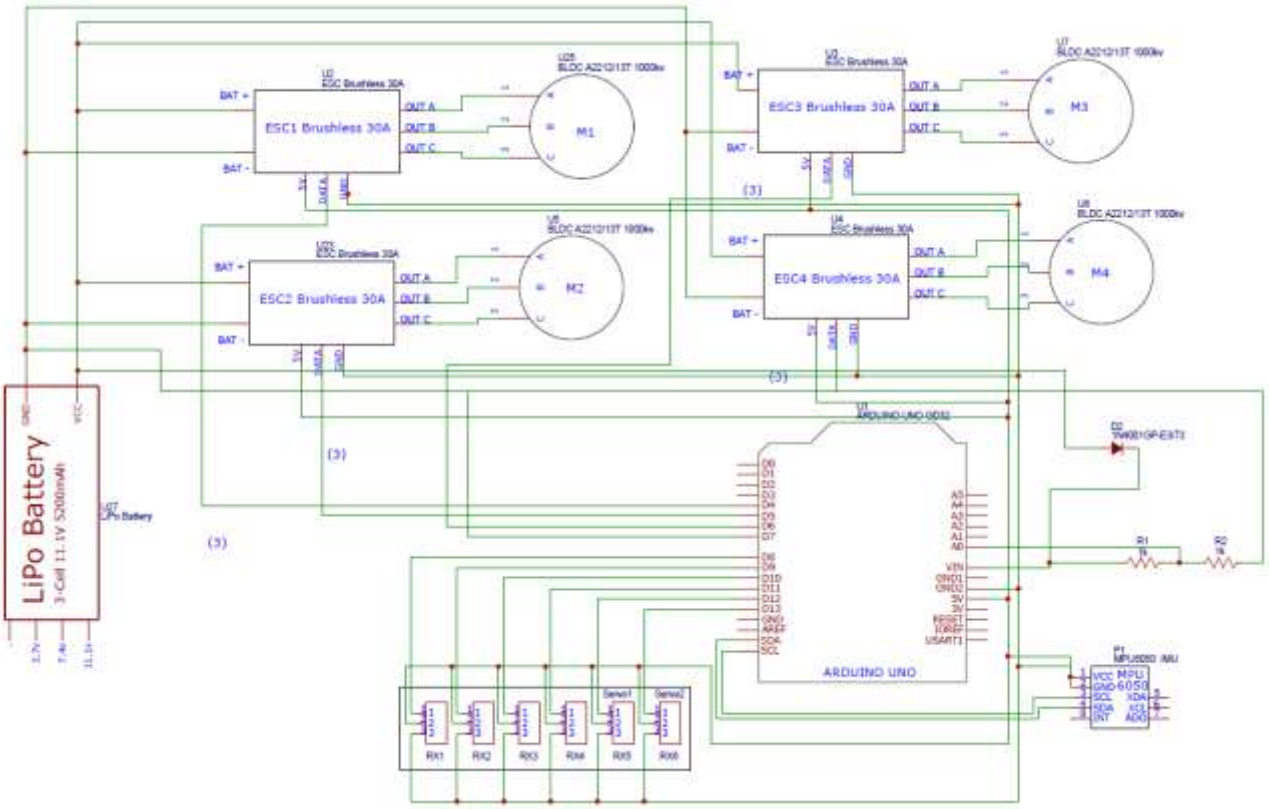
To validate the receiver functionality, it's essential to transmit data from the transmitter, where we use joystick inputs at Tx, and then observe the corresponding output on the serial monitor at the receiver's end.



a) b) c)

a) Initial Values of Rx before testing, b) & c) Rx responding to Tx.

FLIGHT CONTROLLER:



Flight Controller Overall Design

Flight Controller Setup Procedure and Code explanation:

Why diode and resistors?

The battery will discharge during the flight, as a result the motors produce less power, i.e., the motors react differently when battery voltage is dropping during flight.

To compensate this, the controller needs a known battery voltage, this is achieved by developing a voltage divider circuit (NOTE: The voltage divider circuit which I have built in the schematic works only for 3 cell LiPo battery of maximum voltage of 12.6 V). For using 4 cells or more battery, you need to re-calculate the resistors (NOTE: The results should be high, but not above 5V of the analog input with a full battery). Connect the resistors to the Vin and ground of the Arduino uno and connect the resistor connection to the analog input A0.

To prevent the peak power that is feeding the ESC's and motors, a diode needs to be placed as shown in the circuit diagram.

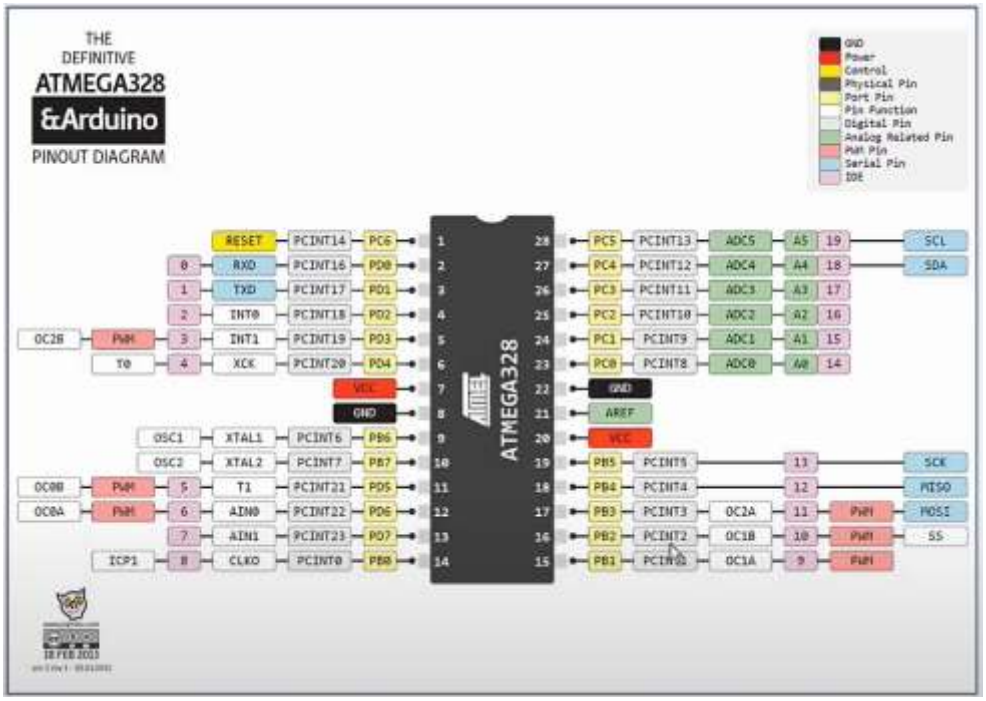
Reading Signals from the receiver by interfacing with Arduino Uno R3:

Beginning with the basic principles of receiver signals, EC's and servos are controlled by pulses that vary 1000 - 2000 microsec. These lengths of the pulses

change as we move the transmitter sticks. The ESC's OFF position is achieved with a 1000 microsec pulse.

Stability is achieved by making around 250 corrections per second in the ESC's. These corrections are also called as REFRESH RATE of the controller. (i.e., ESC's are corrected every 4msec).

Referring to the Datasheet of Atmel Atmega328P;



The use of interrupt registers is given in the below code;

```
PCICR |= (1 << PCIE0);    // set PCIE0 to enable PCMSK0 scan
PCMSK0 |= (1 << PCINT0);  // set PCINT0 (digital input 8) to trigger an interrupt on state change
PCMSK0 |= (1 << PCINT1);  // set PCINT1 (digital input 9)to trigger an interrupt on state change
PCMSK0 |= (1 << PCINT2);  // set PCINT2 (digital input 10)to trigger an interrupt on state change
PCMSK0 |= (1 << PCINT3);  // set PCINT3 (digital input 11)to trigger an interrupt on state change
```

From Chapter 13 of the Atmel Atmeg328P datasheet, under the EXTERNAL INTERRUPT's, we will be using Pin Change Interrupts (PCINT).

The need of interrupt in between the program is for the refresh rate of the controller. When the state of digital pin changes, i.e., receiver outputs as inputs to the flight controller using digital pins D8, D9, D10, D11.

Pin Change Interrupt (PCI0) will trigger if any enabled PCINT [7:0] pin toggles.

To get the PCINT [7:0] pins work, we need to **setup** the Pin Change Interrupt Enable bit (PCIE) in the Pin Change Interrupt Control Register (PCICR).

13.2.5 PCIFR – Pin Change Interrupt Flag Register

Bit	7	6	5	4	3	2	1	0	
0x1B (0x3B)	–	–	–	–	–	PCIF2	PCIF1	PCIF0	PCIFR
Read/Write	R	R	R	R	R	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7:3 – Reserved**

These bits are unused bits in the ATmega48A/PA/88A/PA/168A/PA/328/P, and will always read as zero.

- **Bit 2 – PCIF2: Pin Change Interrupt Flag 2**

When a logic change on any PCINT[23:16] pin triggers an interrupt request, PCIF2 becomes set (one). If the I-bit in SREG and the PCIE2 bit in PCICR are set (one), the MCU will jump to the corresponding Interrupt Vector. The flag is cleared when the interrupt routine is executed. Alternatively, the flag can be cleared by writing a logical one to it.

- **Bit 1 – PCIF1: Pin Change Interrupt Flag 1**

When a logic change on any PCINT[14:8] pin triggers an interrupt request, PCIF1 becomes set (one). If the I-bit in SREG and the PCIE1 bit in PCICR are set (one), the MCU will jump to the corresponding Interrupt Vector. The flag is cleared when the interrupt routine is executed. Alternatively, the flag can be cleared by writing a logical one to it.

- **Bit 0 – PCIF0: Pin Change Interrupt Flag 0**

When a logic change on any PCINT[7:0] pin triggers an interrupt request, PCIF0 becomes set (one). If the I-bit in SREG and the PCIE0 bit in PCICR are set (one), the MCU will jump to the corresponding Interrupt Vector. The flag is cleared when the interrupt routine is executed. Alternatively, the flag can be cleared by writing a logical one to it.

When PCIE0 bit is set (one) and the I-bit in the Status Register (SREG) is set (one), Pin Change Interrupt 0 is enabled. Therefore, any change in PCINT [7:0] pin will cause an interrupt.

7.3.1 SREG – AVR Status Register

The AVR Status Register – SREG – is defined as:

Bit	7	6	5	4	3	2	1	0	
0x3F (0x5F)	I	T	H	S	V	N	Z	C	SREG
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7 – I: Global Interrupt Enable**

The Global Interrupt Enable bit must be set for the interrupts to be enabled. The individual interrupt enable control is then performed in separate control registers. If the Global Interrupt Enable Register is cleared, none of the interrupts are enabled independent of the individual interrupt enable settings. The I-bit is cleared by hardware after an interrupt has occurred, and is set by the RETI instruction to enable subsequent interrupts. The I-bit can also be set and cleared by the application with the SEI and CLI instructions, as described in the instruction set reference.

Focusing only on I- bit,

As per the above picture, Bit 7 – I: Global Interrupt Enable:

- Global Interrupt Enable must be set to “one” for the interrupts to be enabled.
- The “Bit-I” is set by RETI instruction to enable subsequent interrupts. (RETI – Return from Interrupt)
- The “Bit-I” is cleared by hardware after an interrupt has occurred.

- The “Bit-I” can also be set & cleared by the application with the SEI and CLI instructions.

(Refer Page 14 – Reset & Interrupt Handling, 57 – Interrupts table, 280 – Memory programming, from the datasheet)

- When using SEI instruction to enable interrupts, the instructions following SEI will be executed before any pending interrupts.
- When using CLI instruction to disable interrupts, the interrupts will be immediately disabled. No interrupt will be executed after the CLI instruction, even if it occurs simultaneously with the CLI instruction.
- We can choose which pin can cause interrupt by using the Pin Change Mask Register (PCMSK0).
- For reading our receiver inputs, we need to set the PCINT0 to PCINT3 in the PCMSK0 register.

13.2.8 PCMSK0 – Pin Change Mask Register 0

Bit	7	6	5	4	3	2	1	0	
(0x6B)	PCINT7	PCINT6	PCINT5	PCINT4	PCINT3	PCINT2	PCINT1	PCINT0	PCMSK0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7:0 – PCINT[7:0]: Pin Change Enable Mask 7...0**

Each PCINT[7:0] bit selects whether pin change interrupt is enabled on the corresponding I/O pin. If PCINT[7:0] is set and the PCIE0 bit in PCICR is set, pin change interrupt is enabled on the corresponding I/O pin. If PCINT[7:0] is cleared, pin change interrupt on the corresponding I/O pin is disabled.

- Each PCINT [7:0] bit selects whether pin change interrupt is enabled on the corresponding I/O pin.
- If PCINT [7:0] is set and PCIE0 bit in PCICR is set, pin change interrupt is enabled on the corresponding I/O pins.
- If PCINT [7:0] is cleared, pin change interrupt on the corresponding I/O pins is disabled.

Therefore, we need to use PCINT0 to PCINT3 since we have 4 inputs from the receiver through digital pins, i.e., D8, D9, D10, D11.

To use Interrupts referring from the Interrupt Vector Table:

To keep interrupt service routine as short as possible, we use the command “PINB && B00000001” (also referred/ similar to digital read).

The need of ISR (Interrupt Service Routine) to be kept short is that it won’t mess-up the whole program.

From chapter 14, I/O Ports;

In register description,

14.4.4 PINB – The Port B Input Pins Address[1]

Bit	7	6	5	4	3	2	1	0	
0x03 (0x23)	PINB7	PINB6	PINB5	PINB4	PINB3	PINB2	PINB1	PINB0	PINB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	

```
//This routine is called every time input 8, 9, 10 or 11 changed state
ISR(PCINT0_vect){
  //Channel 1=========================================
  if(last_channel_1 == 0 && PINB & B00000001 ){          //Input 8 changed from 0 to 1
    last_channel_1 = 1;                                  //Remember current input state
    timer_1 = micros();                                  //Set timer_1 to micros()
  }
  else if(last_channel_1 == 1 && !(PINB & B00000001)){   //Input 8 changed from 1 to 0
    last_channel_1 = 0;                                  //Remember current input state
    receiver_input_channel_1 = micros() - timer_1;       //Channel 1 is micros() - timer_1
  }
}
```

The Subroutine is executed every time, the digital pins D8 to D11 changes state.

From line 2, we are measuring the time between pulse rising and falling edges. [For 1st receiver channel].

PINB && B00000001 = Checks Port B0 bit is high or low.

If it is high/ enabled, if statement will be activated, if the pulse is rising.

And elseif statement will be triggered if pulse is falling.

The whole program is written 3 more times for another 3 different receiver inputs D9 to D11.

```
//This routine is called every time input 8, 9, 10 or 11 changed state
ISR(PCINT0_vect){
  //Channel 1=========================================
  if(last_channel_1 == 0 && PINB & B00000001 ){         //Input 8 changed from 0 to 1
    last_channel_1 = 1;                                 //Remember current input state
    timer_1 = micros();                                 //Set timer_1 to micros()
  }
  else if(last_channel_1 == 1 && !(PINB & B00000001)){  //Input 8 changed from 1 to 0
    last_channel_1 = 0;                                 //Remember current input state
    receiver_input_channel_1 = micros() - timer_1;      //Channel 1 is micros() - timer_1
  }
  //Channel 2=========================================
  if(last_channel_2 == 0 && PINB & B00000010 ){         //Input 9 changed from 0 to 1
    last_channel_2 = 1;                                 //Remember current input state
    timer_2 = micros();                                 //Set timer_2 to micros()
  }
  else if(last_channel_2 == 1 && !(PINB & B00000010)){  //Input 9 changed from 1 to 0
    last_channel_2 = 0;                                 //Remember current input state
    receiver_input_channel_2 = micros() - timer_2;      //Channel 2 is micros() - timer_2
  }
  //Channel 3=========================================
  if(last_channel_3 == 0 && PINB & B00000100 ){         //Input 10 changed from 0 to 1
    last_channel_3 = 1;                                 //Remember current input state
    timer_3 = micros();                                 //Set timer_3 to micros()
  }
  else if(last_channel_3 == 1 && !(PINB & B00000100)){  //Input 10 changed from 1 to 0
    last_channel_3 = 0;                                 //Remember current input state
    receiver_input_channel_3 = micros() - timer_3;      //Channel 3 is micros() - timer_3
  }
  //Channel 4=========================================
  if(last_channel_4 == 0 && PINB & B00001000 ){         //Input 11 changed from 0 to 1
    last_channel_4 = 1;                                 //Remember current input state
    timer_4 = micros();                                 //Set timer_4 to micros()
  }
  else if(last_channel_4 == 1 && !(PINB & B00001000)){  //Input 11 changed from 1 to 0
    last_channel_4 = 0;                                 //Remember current input state
    receiver_input_channel_4 = micros() - timer_4;      //Channel 4 is micros() - timer_4
  }
}
```

For channel 1 = B00000001
2 = B00000010
3 = B00000100
4 = B00001000

Last line of the “elseif” part implies calculation of the “length of the pulse”.

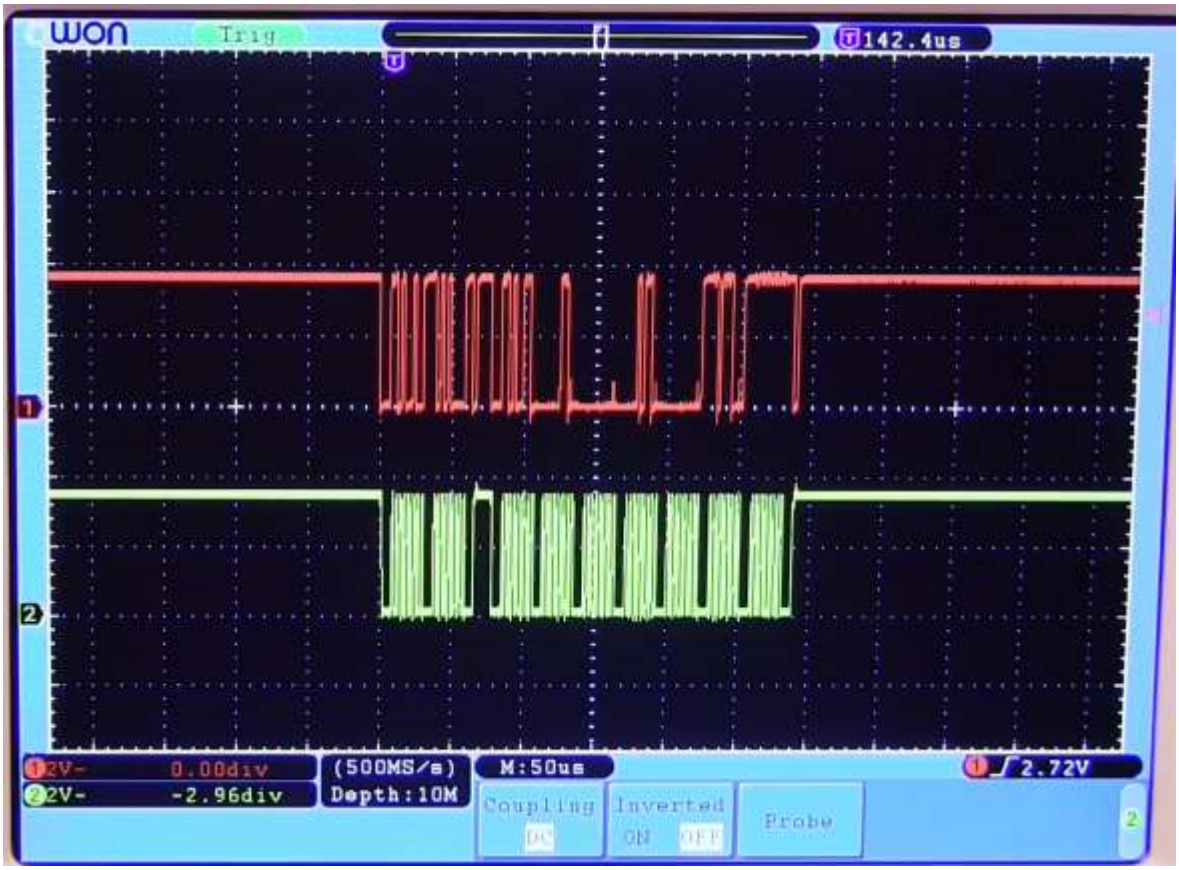
Optimizing the Code for ESC’s:

As per the above discussions, the ESCs are triggered by 1000 - 2000 microsec. And the refresh rate of the ESC pulse/ quadcopter controller is 250 Hz.

The main program of the fight controller includes the steps,

- The receiver inputs are collected via an Interrupt Subroutine. (which will be running in background).

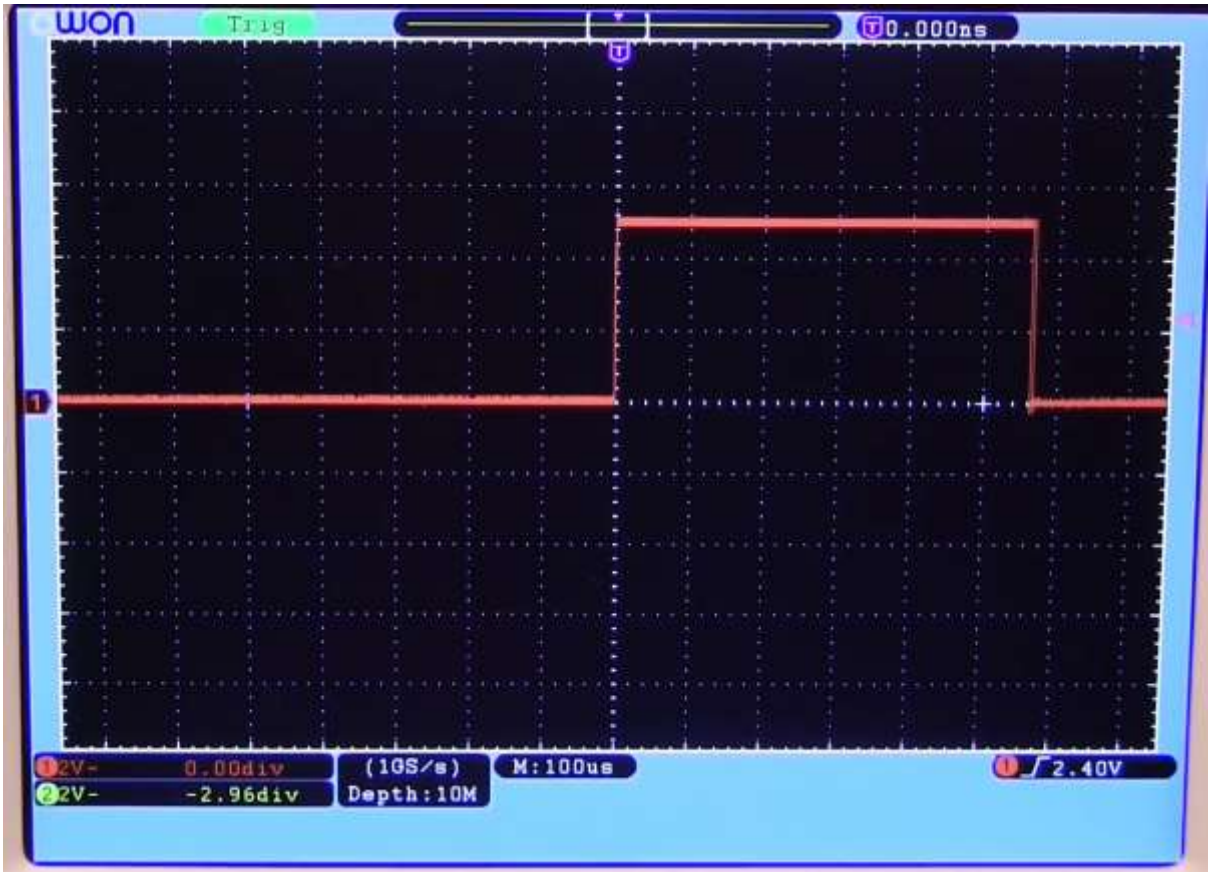
- The main loop of the program involves various steps:
    1. Read the gyro angular rate data.
    2. Calculate the PID corrections.
    3. Calculate the pulse for every ESC.
    4. Compensate every pulse for voltage drop.
    5. Send the calculated pulses to the ESC's.



The above image shows you the I2C data transfer between the Arduino and the gyro, that the slope is set to 50 microsec/division. The total time it takes to transfer the data is approximately 300 microsec.

This pulse represents the calculation time for all the calculations needed to get the quadcopter to fly.

Before the calculations the digital output is set to high, after the calculations the output is set to low. This creates a scope as shown below.



- The calculations take approximately 600 microseconds.
- Our program loop at 250Hz gives 4000 microsec. of available time.
- Reading the gyro angular rate data takes 300 microsec.
- Calculations take 600 microsec.
- Therefore, (4000 - 300 - 600) microsec. = 3100 microsec.

Now, we know that the maximum pulse length for the ESC is 2000 microsec., So there is a plenty of time to create the pulse for the ESCs after the calculations.

Before we use the digital pins for outputs, they need to be declared as outputs. And normally this is done with the command pin mode.

```
sketch_apr17b §
void setup() {
  DDRB |= B00110000;      //Configure digital poort 12 and 13 as output
  //pinMode(12, OUTPUT);
  //pinMode(13, OUTPUT);
}

void loop() {
  digitalWrite(12, HIGH);   //Set digital output 12 high.
  digitalWrite(12, LOW);    //Set digital output 12 low.
  delay(4);                 //Wait for 4 milli seconds.
}
```

We can have another efficient way to do this, the 1st line of the code sets bits into data direction register of Port B. (The result is exactly the same as 2nd and 3rd pinMode lines).

14.4.2 PORTB – The Port B Data Register

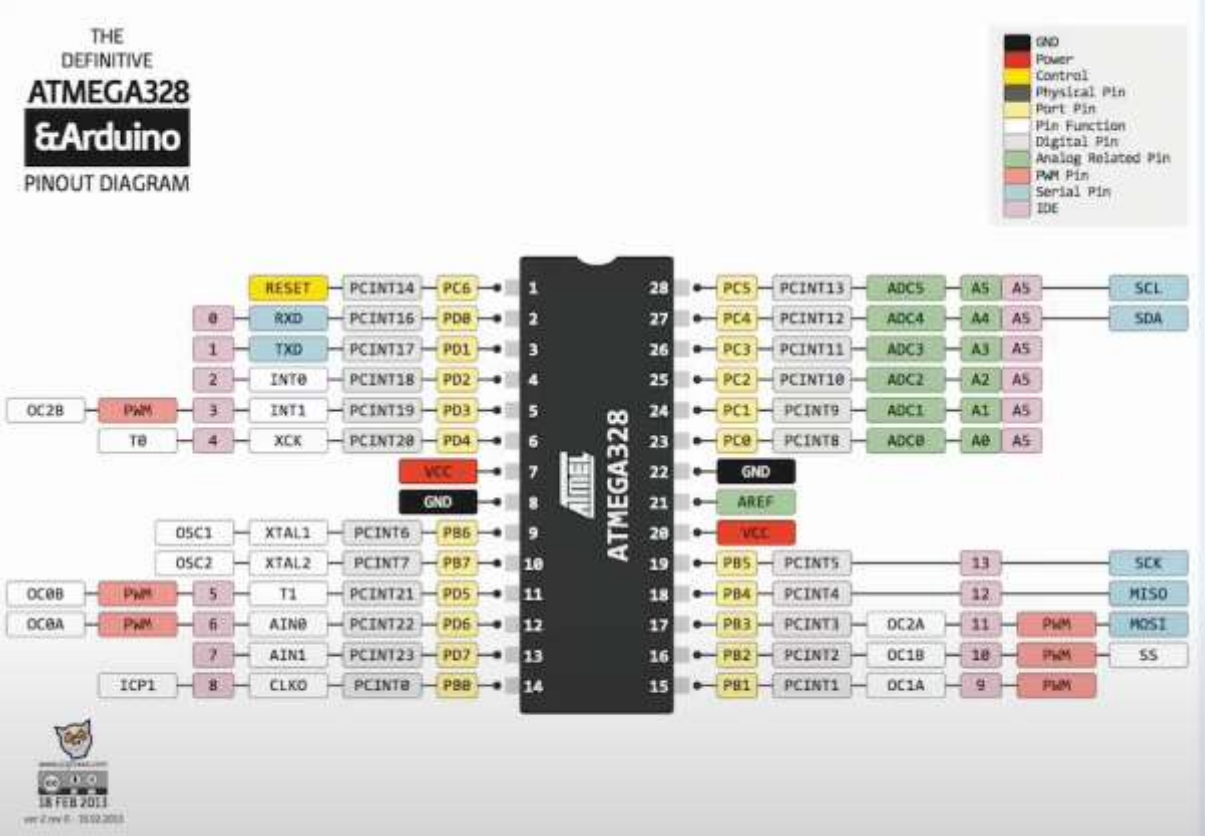
Bit	7	6	5	4	3	2	1	0	
0x05 (0x25)	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0	PORTB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

14.4.3 DDRB – The Port B Data Direction Register

Bit	7	6	5	4	3	2	1	0	
0x04 (0x24)	DDB7	DDB6	DDB5	DDB4	DDB3	DDB2	DDB1	DDB0	DDRB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

As per the above image, the initial valve of the DDRB register bits is 0, (This means all the ports are set to inputs by default).

By setting the DDB4 and DDB5 bit the corresponding ports are defined as outputs.



From the above pin out diagram, we can see that the PB4 and PB5 ports correspond to outputs 12 and 13 respectively of the Arduino uno.

```
void setup() {
  DDRB |= B00110000;      //Configure digital poort 12 and 13 as output
  //pinMode(12, OUTPUT);
  //pinMode(13, OUTPUT);
}

void loop() {
  digitalWrite(12, HIGH);   //Set digital output 12 high.
  digitalWrite(12, LOW);    //Set digital output 12 low.
  delay(4);                 //Wait for 4 milli seconds.
}
```

```
Binaire sketch-grootte: 942 bytes (van een 32.256-byte maximum)
```

After compiling the code, the size is 942 bytes, as shown above.

```
void setup() {
  //DDRB |= B00110000;      //Configure digital poort 12 and 13 as output
  pinMode(12, OUTPUT);
  pinMode(13, OUTPUT);
}

void loop() {
  digitalWrite(12, HIGH);   //Set digital output 12 high.
  digitalWrite(12, LOW);    //Set digital output 12 low.
  delay(4);                 //Wait for 4 milli seconds.
}
```

```
Compileren voltooid

Binaire sketch-grootte: 1.078 bytes (van een 32.256-byte maximum)
```

When compiling the code with the pinMode command, the size is 1078 bytes. This is caused by the extra lines that creates the pinMode commands.

Similarly, the size increases while using the digitalWrite commands.

Therefore, we update the code as shown below,

```
void setup() {
  DDRB |= B00110000;      //Configure digital port 12 and 13 as output
}

void loop() {
  PORTB |= B00110000;
  PORTB &= B11001111;
  //digitalWrite(12, HIGH);   //Set digital output 12 high.
  //digitalWrite(12, LOW);    //Set digital output 12 low.
  delay(4);                 //Wait for 4 milli seconds.
}
```

```
Binaire sketch-grootte: 680 bytes (van een 32.256-byte maximum)
```

Now, the receiver input program, which we used earlier is updated in the interrupt routine to optimize the performance, the updated code in given below.

```
//This routine is called every time input 8, 9, 10 or 11 changed state
ISR(PCINT0_vect){
  current_time = micros();
  //Channel 1=========================================
  if(PINB & B00000001){                                        //Is input 8 high?
    if(last_channel_1 == 0){                                   //Input 8 changed from 0 to 1
      last_channel_1 = 1;                                      //Remember current input state
      timer_1 = current_time;                                  //Set timer_1 to current_time
    }
  }
  else if(last_channel_1 == 1){                                //Input 8 is not high and changed from 1 to 0
    last_channel_1 = 0;                                        //Remember current input state
    receiver_input_channel_1 = current_time - timer_1;         //Channel 1 is current_time - timer_1
  }
  //Channel 2=========================================
  if(PINB & B00000010 ){                                       //Is input 9 high?
    if(last_channel_2 == 0){                                   //Input 9 changed from 0 to 1
      last_channel_2 = 1;                                      //Remember current input state
      timer_2 = current_time;                                  //Set timer_2 to current_time
    }
  }
  else if(last_channel_2 == 1){                                //Input 9 is not high and changed from 1 to 0
    last_channel_2 = 0;                                        //Remember current input state
    receiver_input_channel_2 = current_time - timer_2;         //Channel 2 is current_time - timer_2
  }
  //Channel 3=========================================
  if(PINB & B00000100 ){                                       //Is input 10 high?
    if(last_channel_3 == 0){                                   //Input 10 changed from 0 to 1
      last_channel_3 = 1;                                      //Remember current input state
      timer_3 = current_time;                                  //Set timer_3 to current_time
    }
  }
  else if(last_channel_3 == 1){                                //Input 10 is not high and changed from 1 to 0
    last_channel_3 = 0;                                        //Remember current input state
    receiver_input_channel_3 = current_time - timer_3;         //Channel 3 is current_time - timer_3

  }
  //Channel 4=========================================
  if(PINB & B00001000 ){                                       //Is input 11 high?
    if(last_channel_4 == 0){                                   //Input 11 changed from 0 to 1
      last_channel_4 = 1;                                      //Remember current input state
      timer_4 = current_time;                                  //Set timer_4 to current_time
```

Proportional-Integral-Derivative (PID): -

A PID Controller or a Proportional-Integral-Derivative controller is a control loop feedback mechanism (controller) widely used in industrial control systems. A PID controller calculates an error value as the difference between a measured process variable and a desired setpoint (desired outcome). The controller attempts to minimize the error by adjusting the process through the use of a manipulated variable.

The PID controller's efficacy stems from its three fundamental parameters, often referred to as the three-term control:

• Proportional (P): This component responds to the immediate error magnitude. It's like having a quick reaction. If things are too far off from where they should be, it acts fast to bring them closer. For example, if the temperature in the oven is too low, it cranks up the heat quickly.

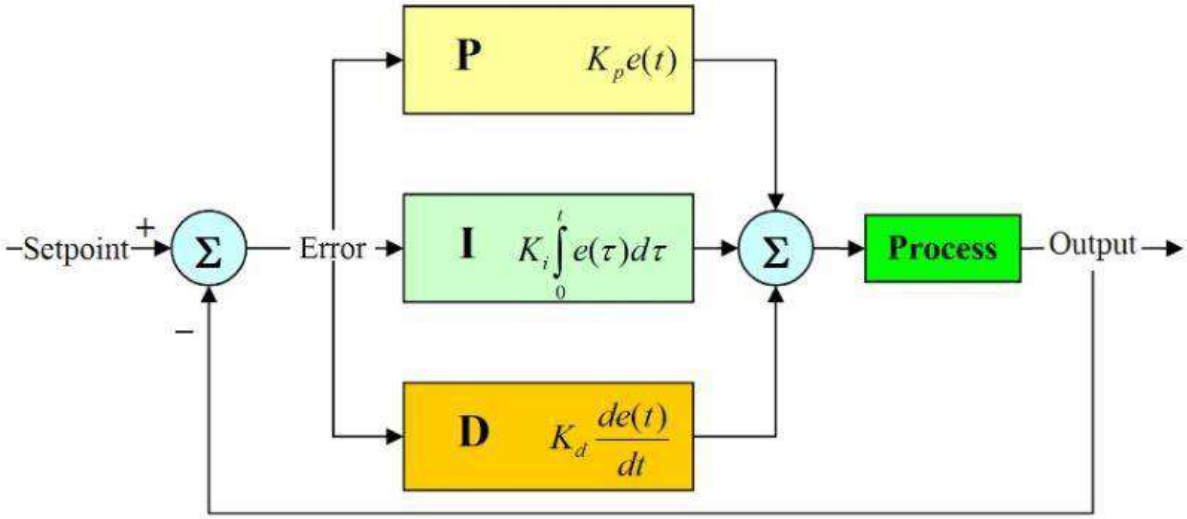
• Integral (I): Reflecting on the accumulation of past errors, this element acts to

address sustained discrepancies.

This part keeps track of past mistakes. If there's been a constant error over time, it gradually adjusts things to fix it. For instance, if the oven temperature has been consistently a bit too high, it will tweak the heat down slowly to compensate.

• Derivative (D): By anticipating future errors through the current rate of change, this factor aids in pre-emptive adjustments. This anticipates future problems. It looks at how fast things are changing and adjusts to prevent big swings. If the temperature in the oven is rising too quickly, it might ease off the heat to avoid overshooting the target.

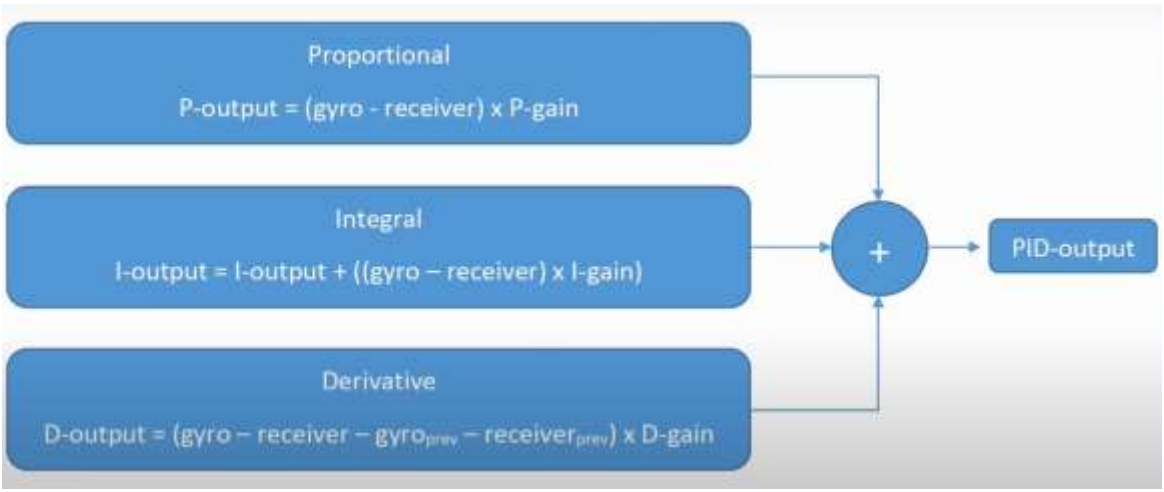
• Error: The error is the amount at which a device isn't doing something right. For example, suppose the robot is located at x=5 but it should be at x=7, then the error is 2.



PID Control System

PID Controller Integration:

A PID controller is integrated into the system to regulate each propeller's rotational speed based on feedback from onboard sensors. This ensures stable flight by maintaining desired altitude, attitude, and trajectory.



The proportional term is given by:

$$P_{out} = K_p * e(t)$$

The integral term is given by:

$$I_{out} = K_i \int_0^t e(\tau)d\tau$$

The Derivative term is given by:

$$D_{out} = K_d \frac{d}{dt} e(t)$$

Final Expression,

$$U(t)=MV(t)= K_p * e(t) + K_i \int_0^t e(\tau)d\tau + K_d \frac{d}{dt} e(t)$$

where,

K_p = Proportional gain, a tuning parameter

K_i = Integral gain, a tuning parameter

K_d = Derivative gain, a tuning parameter

e = error = SP - PV

t = Time or instantaneous time (the present)

τ = variable of integration, takes on values from time 0 to present time t

```
//////////////////////////////////////////////////////////
//PID settings
//////////////////////////////////////////////////////////
float pid_p_gain_roll = 1.4;       //1.3
float pid_i_gain_roll = 0.04;      //0.04
float pid_d_gain_roll = 15.0;      //15.0
int pid_max_roll = 400;
```

The PID gain values are adjustable. Since we don't want the controller to lose its controller, we define a maximum output i.e., pid_max_roll = 400 (You can add your own maximum value), as shown above.

```
//Let's get the current gyro data and scale it to degrees per second for the pid calculations.
gyro_signalen();
//One digit is 17.5 mdps. The maximum output value of the gyro is +/- 32768 (2^15).
//1dps / 17.5mdps = 57.14286.
//To filter the gyro data we use a complementary filter of 20%/80%.
gyro_roll_input = (gyro_roll_input * 0.8) + ((gyro_roll / 57.14286) * 0.2);          //Gyro pid input in deg/sec.
```

The gyro and receiver inputs are in degrees per second.

```
//////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
//Subroutine for calculating pid outputs
//////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
void calculate_pid(){
  //Roll calculations
  pid_error_temp = gyro_roll_input - pid_roll_setpoint; //error = gyro - receiver
  pid_i_mem_roll += pid_i_gain_roll * pid_error_temp;

  if(pid_i_mem_roll > pid_max_roll)pid_i_mem_roll = pid_max_roll;
  else if(pid_i_mem_roll < pid_max_roll * -1)pid_i_mem_roll = pid_max_roll * -1;

  pid_output_roll = pid_p_gain_roll * pid_error_temp + pid_i_mem_roll + pid_d_gain_roll * (pid_error_temp - pid_last_d_error);
  if(pid_output_roll > pid_max_roll)pid_output_roll = pid_max_roll;
  else if(pid_output_roll < pid_max_roll * -1)pid_output_roll = pid_max_roll * -1;

  pid_last_d_error = pid_error_temp;
}
```

In the 1st line, the error between the gyro and receiver is calculated.

The second line is to store the value in the I-register. To prevent the I variable not to go out of control, we need to limit the output to the same output limit as set at the beginning of the program.

Later the complete PID output is calculated.

THE SAME CODE IS WRITTEN FOR PITCH AND YAW ANGLES.

```
//////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
//PID gain and limit settings
//////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
float pid_p_gain_roll = 0;                //Gain setting for the roll P-controller (1.3)
float pid_i_gain_roll = 0;                //Gain setting for the roll I-controller (0.03)
float pid_d_gain_roll = 0;                //Gain setting for the roll D-controller (15)
int pid_max_roll = 400;                   //Maximum output of the PID-controller (+/-)

float pid_p_gain_pitch = pid_p_gain_roll; //Gain setting for the pitch P-controller.
float pid_i_gain_pitch = pid_i_gain_roll; //Gain setting for the pitch I-controller.
float pid_d_gain_pitch = pid_d_gain_roll; //Gain setting for the pitch D-controller.
int pid_max_pitch = pid_max_roll;         //Maximum output of the PID-controller (+/-)

float pid_p_gain_yaw = 3;                 //Gain setting for the pitch P-controller. //4.0
float pid_i_gain_yaw = 0.02;                 //Gain setting for the pitch I-controller. //0.02
float pid_d_gain_yaw = 0;                 //Gain setting for the pitch D-controller.
int pid_max_yaw = 400;                    //Maximum output of the PID-controller (+/-)
```

Change the PID for Roll by using your own values and check whether the quadcopter is working fine.

For pid_d_gain_roll = add in terms of integers, upto 20 or 25. When the quadcopter wiggles a lot it indicates it is the maximum roll values, later reduce the values until you get a stable flight.

For pid_p_gain_roll = Increase in terms of 0.2 and continue the same procedure as you have done for pid_d_gain_roll.

For pid_i_gain_roll = Increase in terms of 0.01 and continue the same procedure as you have done for pid_d_gain_roll.

*(NOTE: Use **Flysky fs T6 Transmitter and Receiver** while working on Arduino Uno Flight controller)*

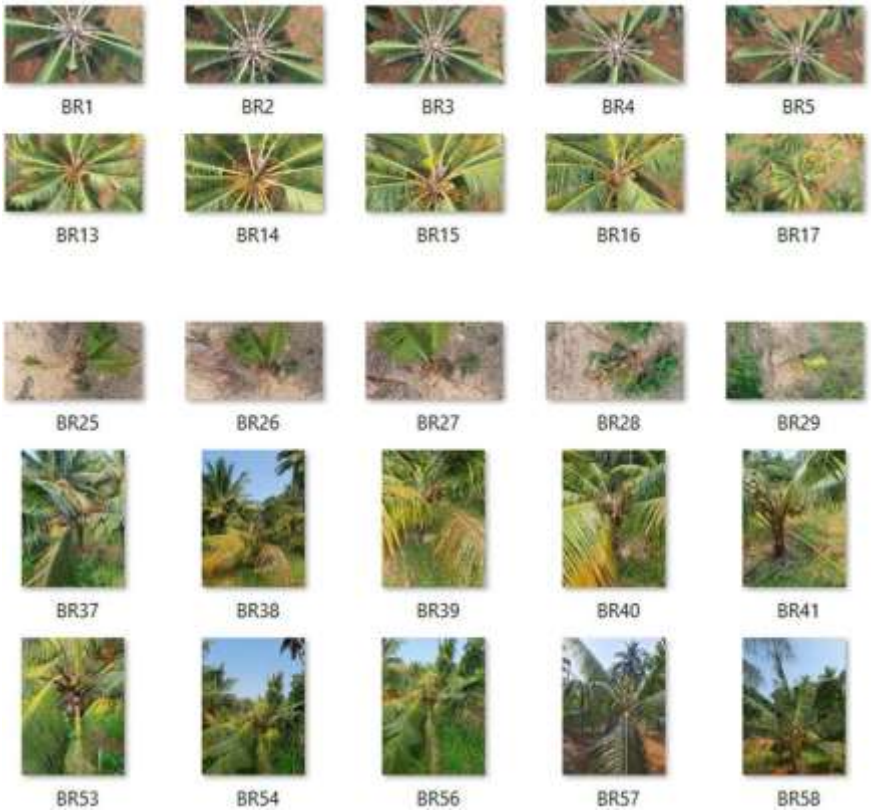
The complete code is linked below in the appendix.

Data Collection

Collaborated with CPRI Kasargod to conduct surveys within coconut and arecanut plantations, observing and documenting disease symptoms and affected areas. This also involved obtaining historical disease records for comprehensive analysis.

SNAPSHOTS:

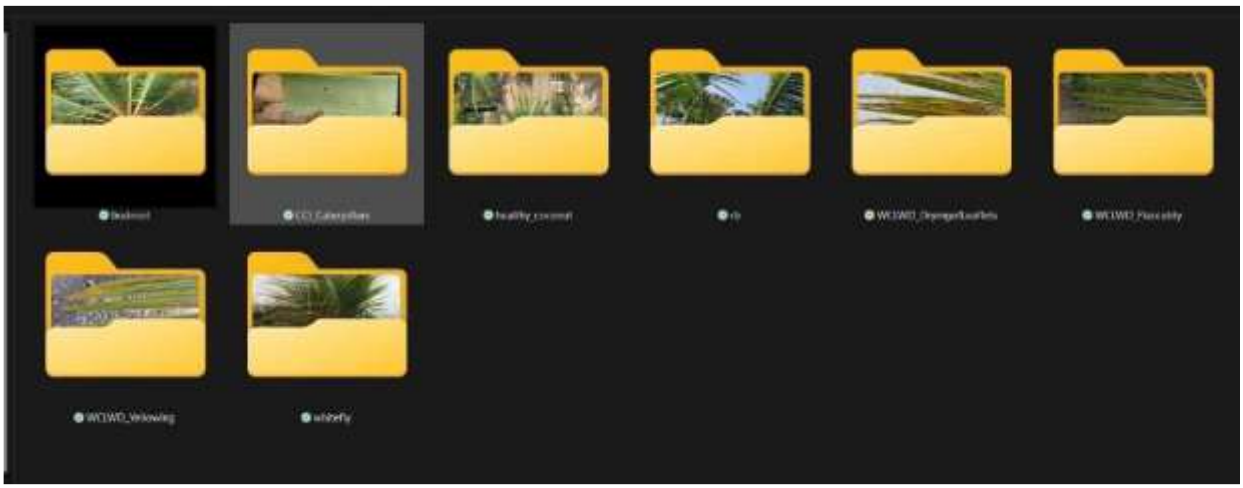
1. COCONUT LEAF DATASET:



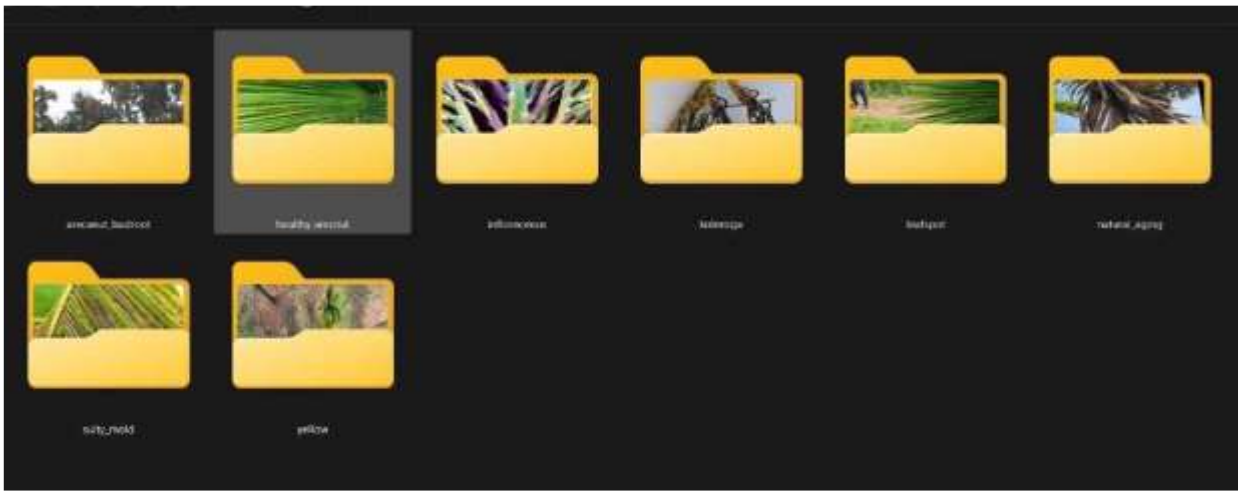
2. ARECANUT LEAF DATASET:

FIELD WORK:

CLASSIFICATIONS:



Coconut Dataset classnames



arecanut Dataset classnames

Modelling

Coconut Dataset model training:

Model Accuracy of coconut dataset

```
Epoch 31 - Loss: 0.05687130111566928
Epoch 32 - Loss: 0.03918246157719385
Epoch 33 - Loss: 0.0714217487793476
Epoch 34 - Loss: 0.05837431561475578 4
Epoch 35 - Loss: 0.03329466758089608
Epoch 36 - Loss: 0.090595507577006
Epoch 37 - Loss: 0.03976782685016847 6
Epoch 38 - Loss: 0.04069817321530766
Epoch 39 - Loss: 0.06868433605464255
Epoch 40 - Loss: 0.05034583677175700 4
Epoch 41 - Loss: 0.03352347967366159
Epoch 42 - Loss: 0.04404909770912107
Epoch 43 - Loss: 0.04749744965510627
Epoch 44 - Loss: 0.03482468182809808
Epoch 45 - Loss: 0.03548935436313284
Epoch 46 - Loss: 0.0344416231693428
Epoch 47 - Loss: 0.02961845703480271 4
Epoch 48 - Loss: 0.04194213321906128
Epoch 49 - Loss: 0.0503088856418829
Epoch 50 - Loss: 0.03693342725815096
Accuracy of the network on the test images: 98 %
```

Arecanut dataset model training:

Model Accuracy of arecanut dataset

Implementation

PSEUDOCODE:

DATA AUGMENTATION AND PREPROCESSING:

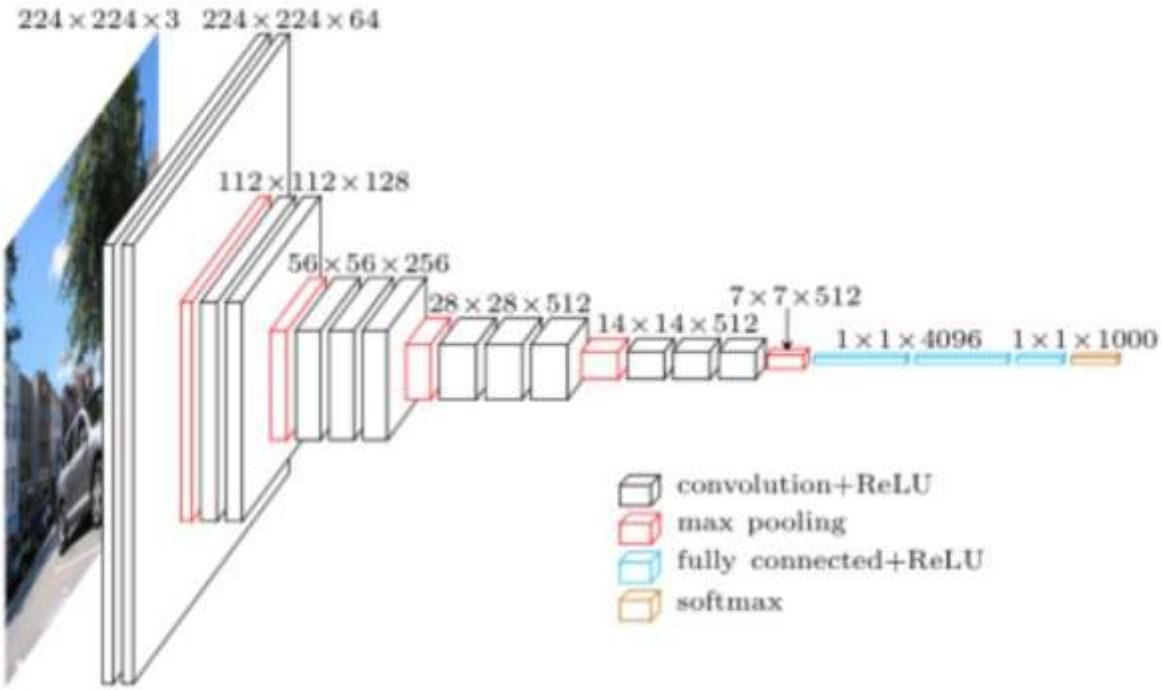
```
#Define transformations for data augmentation for training:
train_transform = Compose ([
        RandomResizedCrop(224),
        RandomHorizontalFlip(),
        ToTensor(),
        Normalize (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])
#Define transformations for testing:
test_transform = Compose([
        Resize((224, 224)),
        ToTensor(),
        Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])
```

```
#Load dataset without specifying train/test directories:
dataset = ImageFolder(root=data_path, transform=train_transform)
#Split dataset into training and testing sets:
train_size = 0.8 * len(dataset)
test_size = len(dataset) - train_size
train_dataset, test_dataset = random_split(dataset, [train_size, test_size])
#Define data loaders:
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)

test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)
```

VGG16 MODEL TRAINING:

```
#Import necessary libraries:

import torchvision.models as models

import torch.nn as nn
import torch.optim as optim
import torch
#Define VGG16 model:
model = models.vgg16(pretrained=True)
#Modify classifier to match the number of classes in your dataset:
num_features = model.classifier[6].in_features
model.classifier[6] = nn.Linear(num_features, len(dataset.classes))
#Define loss function and optimizer:
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.0001) # Lower learning rate
#Move model to GPU if available:
device = "cuda" if torch.cuda.is_available() else "cpu"
model.to(device)
#Train the model:
model.train()
```

```
for epoch in range(100): # Train for more epochs
        running_loss = 0.0
        for images, labels in train_loader:
                Move data to GPU if available
                optimizer.zero_grad()
                outputs = model(images)
                loss = criterion(outputs, labels)

                loss.backward()

                optimizer.step()
                running_loss += loss.item()

                #Print epoch and average loss
```



VGG-16 model

MODEL EVALUATION

```
#Set the model to evaluation mode:
model.eval()
#Initialize variables for accuracy calculation:
correct = 0
total = 0
#Disable gradient computation for evaluation:
with torch.no_grad():
#Iterate over batches in the test_loader:
for images, labels in test_loader:
        #Move data to GPU if available:
        images, labels = images.to(device), labels.to(device)
        #Forward pass to get predictions:
        outputs = model(images)
        #Compute predicted classes:
        _, predicted = torch.max(outputs, 1)
        #Update total number of samples:

        total += labels.size(0)

        #Update number of correctly classified samples:
        correct += (predicted == labels).sum().item()
        #Calculate and print accuracy:
        accuracy = 100 * correct / total

        #print ('Accuracy of the network on the test images: %d %%' % accuracy))
```

Disease detection video is linked in the appendix

2) Advancing Drone Technology through Automation

Objectives:

1. Enhance Autonomous Navigation: Developing an advanced algorithms for precise autonomous navigation, enabling drones to perform complex missions without human intervention, including dynamic path planning.
2. Improve Flight Stability and Control: Integrating sophisticated control systems to enhance the stability, agility, and responsiveness of drones, especially in challenging environmental conditions.
3. Expand Sensor Integration: Incorporating a wide range of sensors, such as Telemetry, Barometer, and GPS-Magnetometer and Compass, to enhance situational awareness and data collection capabilities, allowing drones to perform detailed environmental analysis.

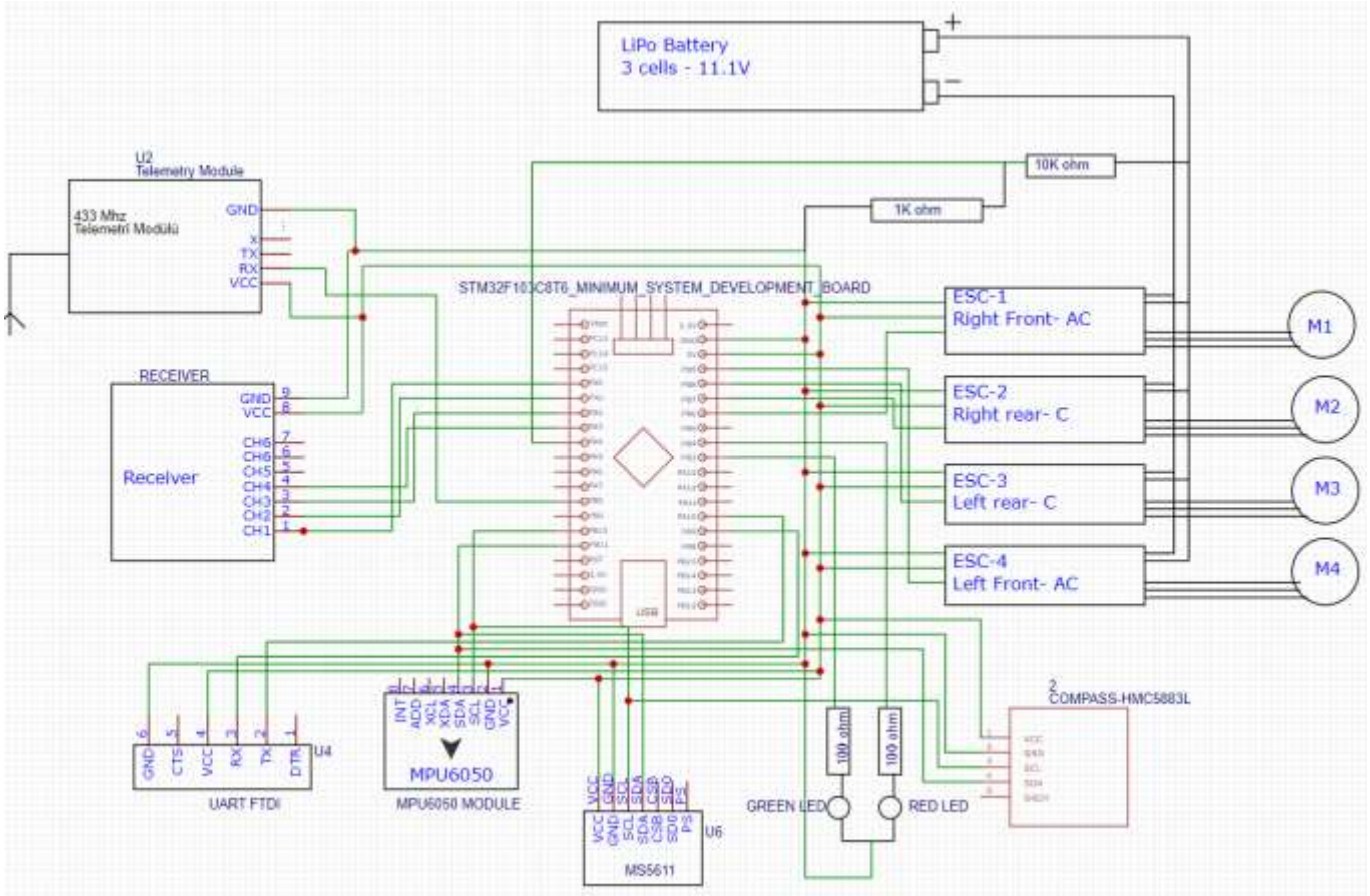
Hardware Requirements for autonomous flight controller:

- STM32F103C8T6
- FTDI FT232RL Programmer board
- MPU-6050 gyro / accelerometer
- GPS module with integrated HMC5883L compass
- MS5611 Barometric Pressure sensor
- Flysky FS-i6X transmitter with IA6B Receiver
- 1/4W Ceramic metal film resistors set (600 PCS)
- Glass fibre prototyping PCB
- 3mm LEDs, red, yellow, green

Telemetry System:

- Arduino UNO + 16x2 LCD keypad Shield
- LCD breakout board with buttons
- Buzzer Module
- APC220 Wireless RF Modules with USB Converter

Flight Controller Circuit:



Programming a STM32F103C8T6 microcontroller

- We can program the STM32 microcontroller by using Arduino IDE (either 1.8.3 or the latest version).
- Open the Arduino IDE and go to tools -> board manager -> Search " 32-bit ARM Cortex M3 " and install it.
- After installing, close the IDE and go to Arduino_STM32 page on GitHub. (link in appendix)
- This is an Arduino add on solution for the STM32 microcontrollers, created by Roger Clak.
- Download the zip file from GitHub.
- Unpack the Arduino_STM32_master directory, copy it and place the complete directory in the hardware directory of the Arduino IDE.

Now the STM32 microcontroller is able to program through Arduino IDE.

Connecting FTDI Programmer to the STM32 board

- Note that the STM32F103C8T6 is a 3.3V microcontroller, despite the fact that some pins are 5V tolerance, connecting to the 5V for the power supply will definitely destroy the board.
- To avoid this, make sure to connect the 5V from the FTDI programmer to the

5V input of the STM32 board.

- General Information:
    1. On the back side of the board, there is a 3.3V voltage regulator, that will provide the 3.3V for the microcontroller.
    2. Note that this voltage regulator cannot provide a whole lot of current.
    3. Therefore, connecting a lot of LEDs for testing might overload the voltage regulator.
- Connect 5V of STM32 board to 5V FTDI Programmer,

  Connect GND to GND,

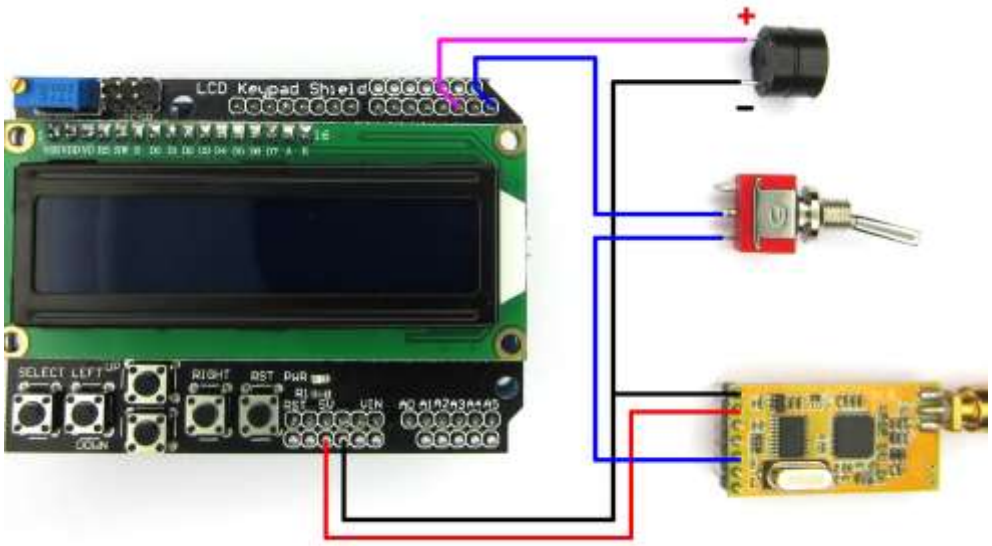
  Connect A9 of the STM32 Board to RX of the FTDI Programmer,

  Connect A10 of the STM32 Board to TX of the FTDI Programmer.

Uploading the Program to STM32 board

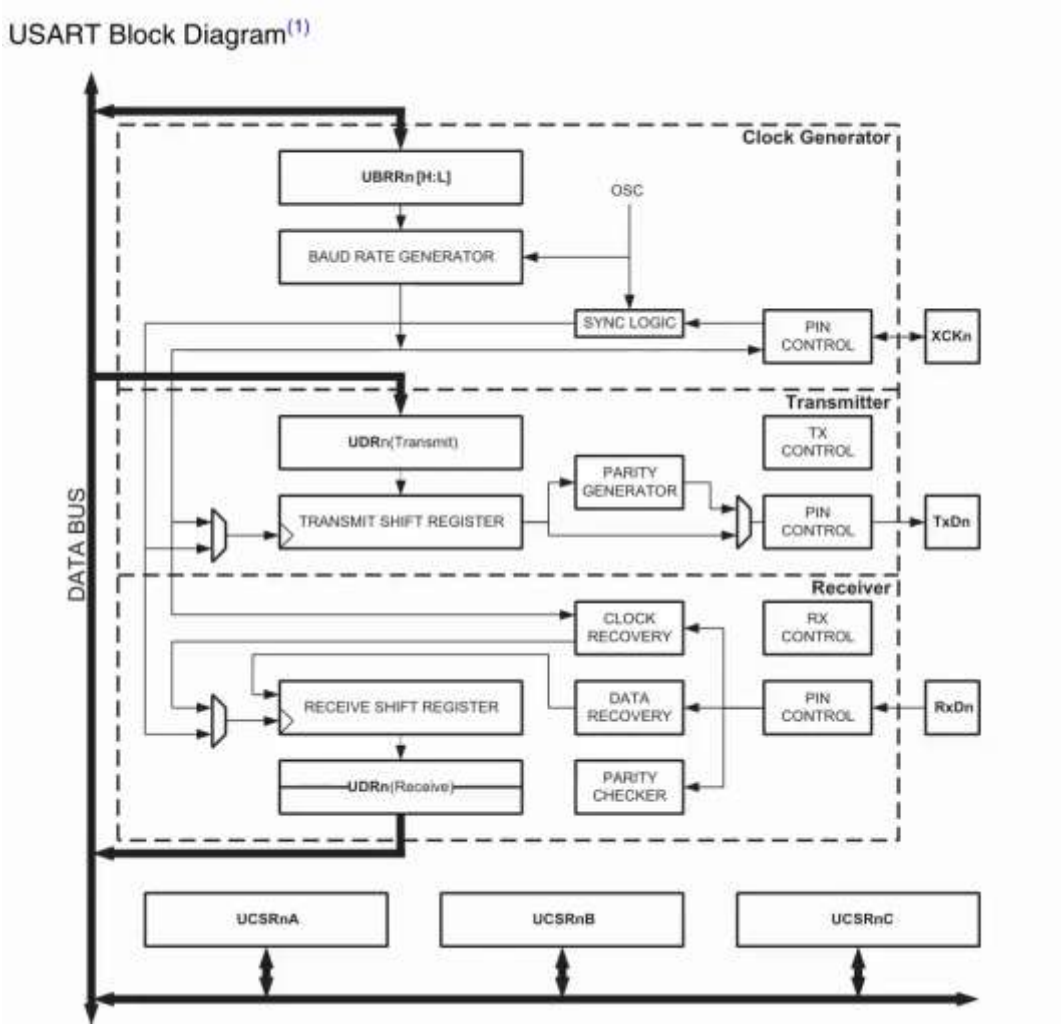
- Open Arduino IDE -> Tools -> Boards -> Select " Generic STM32F103C Series ".
- Go to tools -> Variant -> Select " STM32F103C8 (20k RAM, 64k Flash).
- Tools -> CPU Speed -> Select " 72 MHz (Normal) ".
- Tools -> Upload method -> Select " Serial".
- Finally select the COM port to which the FTDI Programmer is connected.
- To check whether all the setup is correct,
    1. Go to File -> Examples -> A_STM32_Examples -> Digital -> Blink (Open the Blink sketch)
    2. Note that the LED on the STM32 board is connected to pin PC_13.
    3. Therefore," replace all PB1 " in the code to " PC13 " to get the LED blink.
    4. Before each upload, switch the tuff jumper to the right and press reset.
    5. This way the microcontroller enters the program mode.
    6. Upload the program.
    7. To upload again, press the reset button.
    8. When you want to execute the program without changing the program again, upload the code and switch the tuff jumper left, and press reset.

Telemetry connection and coding



Telemetry connection at ground station

The drone holds several variables that hold valuable information's such as battery voltage, flight mode, errors, satellites etc., To show these values in the LCD, we need to transfer the values from the quadcopter to the ground station. This is achieved by using serial communication protocol.



The protocol used here is time based, i.e., after a receive start, timer is needed to simulate clock pulse. Therefore, the baud rates at both the telemetry devices must be same.

Consider a Baud rate of 9600. Here, 9600 baud rate means the low or high state of the data line change 9600 times per second. With every data transfer 8 bits are sent to the receiver. These bits can be anything i.e., all 0's or all 1's or combination of both.

When the data line is not used, it's pulled high (5V). So, for example, if when we send all 1's, the receiver will not be able to identify that anything has been sent, as the data line has never been low.

To overcome this problem, start indication is needed, also called as "Start Bit". The start bit will pull the data line to low to indicate the START. By using the "Stop Bit" the data line is pulled high, so the receiver has a known synchronization point, that is used to reset the clock of the receiver.

```
///If the line starts with $a and if there is a GPS fix we can scan the line for the latitude, longitude and number of satellites.
if (incoming_message[0] == '$' && incoming_message[5] == 'A' && (incoming_message[44] == '1' || incoming_message[44] == '2')) {
  lat_gps_actual = ((int)incoming_message[18] - 48) * (long)10000000;          //Filter the minutes for the GGA line multiplied by 10.
  lat_gps_actual += ((int)incoming_message[20] - 48) * (long)1000000;          //Filter the minutes for the GGA line multiplied by 10.
  lat_gps_actual += ((int)incoming_message[21] - 48) * (long)100000;           //Filter the minutes for the GGA line multiplied by 10.
  lat_gps_actual += ((int)incoming_message[22] - 48) * (long)10000;            //Filter the minutes for the GGA line multiplied by 10.
  lat_gps_actual += ((int)incoming_message[24] - 48) * (long)1000;             //Filter the minutes for the GGA line multiplied by 10.
  lat_gps_actual += ((int)incoming_message[25] - 48) * (long)100;              //Filter the minutes for the GGA line multiplied by 10.
  lat_gps_actual += ((int)incoming_message[26] - 48) * (long)10;               //Filter the minutes for the GGA line multiplied by 10.
  lat_gps_actual /= (long)6;                                                   //To convert the minutes to degrees we need to divide the minutes by 6.
  lat_gps_actual += ((int)incoming_message[17] - 48) * (long)100000000;        //Add the degrees multiplied by 10.
  lat_gps_actual += ((int)incoming_message[16] - 48) * (long)10000000;         //Add the degrees multiplied by 10.
  lat_gps_actual /= 10;                                                        //Divide everything by 10.

  lon_gps_actual = ((int)incoming_message[33] - 48) * (long)10000000;          //Filter the minutes for the GGA line multiplied by 10.
  lon_gps_actual += ((int)incoming_message[34] - 48) * (long)1000000;          //Filter the minutes for the GGA line multiplied by 10.
  lon_gps_actual += ((int)incoming_message[36] - 48) * (long)100000;           //Filter the minutes for the GGA line multiplied by 10.
  lon_gps_actual += ((int)incoming_message[37] - 48) * (long)10000;            //Filter the minutes for the GGA line multiplied by 10.
  lon_gps_actual += ((int)incoming_message[38] - 48) * (long)1000;             //Filter the minutes for the GGA line multiplied by 10.
  lon_gps_actual += ((int)incoming_message[39] - 48) * (long)100;              //Filter the minutes for the GGA line multiplied by 10.
  lon_gps_actual += ((int)incoming_message[40] - 48) * (long)10;               //Filter the minutes for the GGA line multiplied by 10.
  lon_gps_actual /= (long)6;                                                   //To convert the minutes to degrees we need to divide the minutes by 6.
  lon_gps_actual += ((int)incoming_message[30] - 48) * (long)100000000;        //Add the degrees multiplied by 10.
  lon_gps_actual += ((int)incoming_message[31] - 48) * (long)10000000;         //Add the degrees multiplied by 10.
  lon_gps_actual += ((int)incoming_message[32] - 48) * (long)10000000;         //Add the degrees multiplied by 10.
  lon_gps_actual /= 10;                                                        //Divide everything by 10.

  if (incoming_message[28] == 'N')latitude_north = 1;                          //When flying north of the equator the latitude_north variable will be set to
  else latitude_north = 0;                                                     //When flying south of the equator the latitude_north variable will be set to

  if (incoming_message[42] == 'E')longitude_east = 1;                          //When flying east of the prime meridian the longitude_east variable will be se
  else longitude_east = 0;                                                     //When flying west of the prime meridian the longitude_east variable will be se
```

The above code represents storing the actual longitude and latitude values. These values are assigned as 32-bit integer values as latitude and longitude values go up to 10 digits. To send a 32-bit integer, when we only consent 8 bits at a time, we can use the "bitshift left (<<), bitshift right (>>)" operator.

```
YMFC-32_auto_Flight_Controller_ver1b.ino   Receiver.ino   LED_control.ino   calculate_pid.ino   calibration.ino   change_settings.ino   gps.ino   input_capture_mode_handlers.ino   read_compass.ino   read_gps.ino
    if (esc_2 < motor_idle_speed) esc_2 = motor_idle_speed;          //Keep the motors running.
    if (esc_3 < motor_idle_speed) esc_3 = motor_idle_speed;          //Keep the motors running.
    if (esc_4 < motor_idle_speed) esc_4 = motor_idle_speed;          //Keep the motors running.

    if (esc_1 > 2000)esc_1 = 2000;                                   //Limit the esc-1 pulse to 2000us.
    if (esc_2 > 2000)esc_2 = 2000;                                   //Limit the esc-2 pulse to 2000us.
    if (esc_3 > 2000)esc_3 = 2000;                                   //Limit the esc-3 pulse to 2000us.
    if (esc_4 > 2000)esc_4 = 2000;                                   //Limit the esc-4 pulse to 2000us.
  }

  else {
    esc_1 = 1000;                                                    //If start is not 2 keep a 1000us pulse for ess-1.
    esc_2 = 1000;                                                    //If start is not 2 keep a 1000us pulse for ess-2.
    esc_3 = 1000;                                                    //If start is not 2 keep a 1000us pulse for ess-3.
    esc_4 = 1000;                                                    //If start is not 2 keep a 1000us pulse for ess-4.
  }


  TIMER4_BASE->CCR1 = esc_1;                                         //Set the throttle receiver input pulse to the ESC 1 output pulse.
  TIMER4_BASE->CCR2 = esc_2;                                         //Set the throttle receiver input pulse to the ESC 2 output pulse.
  TIMER4_BASE->CCR3 = esc_3;                                         //Set the throttle receiver input pulse to the ESC 3 output pulse.
  TIMER4_BASE->CCR4 = esc_4;                                         //Set the throttle receiver input pulse to the ESC 4 output pulse.
  TIMER4_BASE->CNT = 5000;                                           //This will reset timer 4 and the ESC pulses are directly created.

  send_telemetry_data();                                             //Send telemetry data to the ground station.

  //!!!
```

NOTE: It's not possible to send multiple bytes in every main program loop. As this will take more than 4000 microsec maximum. Therefore, to overcome this problem only 1 byte is sent every program loop.

In every program loop, "send_telemetry_data" subroutine is executed.

```
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
//This part sends the telemetry data to the ground station.
//The output for the serial monitor is PB0. Protocol is 1 start bit, 8 data bits, no parity, 1 stop bit.
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
void send_telemetry_data(void) {
  telemetry_loop_counter++;                                          //Increment the telemetry_loop_counter variable.
  if (telemetry_loop_counter == 1)telemetry_send_byte = '1';         //Send a 1 as start signature.
  if (telemetry_loop_counter == 2)telemetry_send_byte = '8';         //Send a 8 as start signature.
  if (telemetry_loop_counter >= 3) {
    check_byte = 0;
    telemetry_send_byte = error;                   //Send the error as a byte.
  }
  if (telemetry_loop_counter == 4)telemetry_send_byte = flight_mode;          //Send the flight mode as a byte.
  if (telemetry_loop_counter == 5)telemetry_send_byte = battery_voltage * 10; //Send the battery voltage as a byte.
  if (telemetry_loop_counter == 6) {
    telemetry_buffer_byte = temperature;                                      //Store the temperature as it can change during the next loop.
    telemetry_send_byte = telemetry_buffer_byte;                              //Send the first 8 bytes of the temperature variable.
  }
  if (telemetry_loop_counter == 7)telemetry_send_byte = telemetry_buffer_byte >> 8;   //Send the last 8 bytes of the temperature variable.
  if (telemetry_loop_counter == 8)telemetry_send_byte = angle_roll + 100;             //Send the roll angle as a byte. Adding 100 prevents negative numbers.
  if (telemetry_loop_counter == 9)telemetry_send_byte = angle_pitch + 100;            //Send the pitch angle as a byte. Adding 100 prevents negative numbers.
  if (telemetry_loop_counter == 10)telemetry_send_byte = start;                       //Send the error as a byte.
  if (telemetry_loop_counter == 11) {
    if (start == 2) {                                                         //Only send the altitude when the quadcopter is flying.
      telemetry_buffer_byte = 1000 + ((ground_pressure - actual_pressure) * 0.0043);   //Calculate the altitude and add 1000 to prevent negative numbers.
    }
    else {
      telemetry_buffer_byte = 1000;                                           //Send and altitude of 0 meters if the quadcopter isn't flying.
    }
    telemetry_send_byte = telemetry_buffer_byte;                              //Send the first 8 bytes of the altitude variable.
  }
  if (telemetry_loop_counter == 12)telemetry_send_byte = telemetry_buffer_byte >> 8;  //Send the last 8 bytes of the altitude variable.

  if (telemetry_loop_counter == 13) {
    telemetry_buffer_byte = 1500 + takeoff_throttle;                          //Store the take-off throttle as it can change during the next loop.
    telemetry_send_byte = telemetry_buffer_byte;                              //Send the first 8 bytes of the take-off throttle variable.
  }
  if (telemetry_loop_counter == 14)telemetry_send_byte = telemetry_buffer_byte >> 8;  //Send the last 8 bytes of the take-off throttle variable.
```

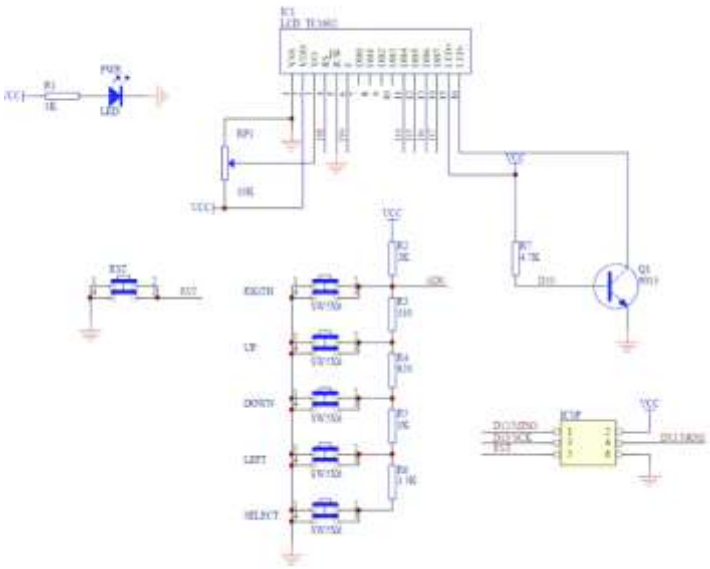
From the above code, the 1st two loops a signature is sent so the ground station can detect the start of the data stream. After that, during every loop a single byte is sent.

At the receiver side, the baud rate is assigned same as in the transmitter side. After that the bytes are received automatically in a rotating buffer via interrupts. We can check, if there are new bytes available with "serial_available" function. Later we can read the received bytes with the "serial_read" function. Then, the remaining code is executed as shown below.

```
  telemetry_buffer_byte = adjustable_setting_1 * 100;                 //Store the adjustable setting 1 as it can change during the next loop.
  telemetry_send_byte = telemetry_buffer_byte;                        //Send the first 8 bytes of the adjustable setting 1.
  }
  if (telemetry_loop_counter == 29)telemetry_send_byte = telemetry_buffer_byte >> 8;  //Send the next 8 bytes of the adjustable setting 1.
  if (telemetry_loop_counter == 30) {
    telemetry_buffer_byte = adjustable_setting_2 * 100;               //Store the adjustable setting 2 as it can change during the next loop.
    telemetry_send_byte = telemetry_buffer_byte;                      //Send the first 8 bytes of the adjustable setting 2.
  }
  if (telemetry_loop_counter == 31)telemetry_send_byte = telemetry_buffer_byte >> 8;  //Send the next 8 bytes of the adjustable setting 2.
  if (telemetry_loop_counter == 32) {
    telemetry_buffer_byte = adjustable_setting_3 * 100;               //Store the adjustable setting 3 as it can change during the next loop.
    telemetry_send_byte = telemetry_buffer_byte;                      //Send the first 8 bytes of the adjustable setting 3.
  }
  if (telemetry_loop_counter == 33)telemetry_send_byte = telemetry_buffer_byte >> 8;  //Send the next 8 bytes of the adjustable setting 3.


  if (telemetry_loop_counter == 34)telemetry_send_byte = check_byte;  //Send the check byte.


  //After 125 loops the telemetry_loop_counter variable is reset. This way the telemetry data is send every half second.
  if (telemetry_loop_counter == 125)telemetry_loop_counter = 0;       //After 125 loops reset the telemetry_loop_counter variable

  //Send the telemetry_send_byte via the serial protocol via output PB0.
  //Send a start bit first.
  if (telemetry_loop_counter <= 34) {
    check_byte ^= telemetry_send_byte;
    GPIOB_BASE->DOUT = 0b1 << 16;                                     //Reset output PB0 to 0 to create a start bit.
    delayMicroseconds(104);                                           //Delay 104us (1s/9600bps)
    for (telemetry_bit_counter = 0; telemetry_bit_counter < 8; telemetry_bit_counter++) {   //Create a loop form every bit in the
      if (telemetry_send_byte >> telemetry_bit_counter & 0b1) GPIOB_BASE->DOUT = 0b1 << 0;   //If the specific bit is set, set output PB0 to 1;
      else GPIOB_BASE->DOUT = 0b1 << 16;                              //If the specific bit is not set, reset output PB0 to 0;
      delayMicroseconds(104);                                         //Delay 104us (1s/9600bps)
    }
    //Send a stop bit.
    GPIOB_BASE->DOUT = 0b1 << 0;                                      //Set output PB0 to 1;
  }
}
```

Regarding the key inputs, the keys are connected via resistors to one analog input.

So, when a key is pressed a voltage on the analog input changes**.**



```
void setup() {
  //Let's setup a interrupt that will check the key inputs on the analog input A0.
  TCCR2A = 0;                                          //Make sure that the TCCR2A register is set to zero
  TCCR2B = 0;                                          //Make sure that the TCCR2A register is set to zero
  TIMSK2 |= (1 << OCIE2A);                             //Set the interrupt enable bit OCIE2A in the TIMSK2 register
  TCCR2B |= (1 << CS22|1 << CS21|1 << CS20);           //Set the CS20, CS21 and CS22 bit in the TCCR0 register to set the prescaler to 1024
  OCR2A = 249;                                         //The compare register is set to 249 => 1ms / (1s / (16.000.000Hz / 1024)) - 1 = 249
  TCCR2A |= (1 << WGM21);                              //Set counter 2 to CTC (clear timer on compare) mode
```

The interrupt checks the keys every few microseconds. This helps in creating a good responsive user interface.

Working on GPS requires “u-center GNSS Evaluation Software” from U-Blox is a powerful tool developed by u-blox for the evaluation, configuration, and testing of GNSS (Global Navigation Satellite System) receivers. This software is particularly useful for users working with u-blox GNSS products, providing a comprehensive environment to monitor and analyze GNSS data in real-time. *(To work on this, and continuing the autonomous flight, link is provided in the appendix)*

The Gyro – AUTO LEVELLING

(This explanation about auto levelling is used for Arduino Uno Flight controller code. In STM32 Flight Controller, it plays the same role with few differences in the program)

The MPU-6050 (Motion Processing Unit) is a popular 6-axis motion tracking device, often used in applications that require precise measurement of motion and orientation. It integrates a 3-axis gyroscope and a 3-axis accelerometer on a single chip.

From the Datasheet, we see that,

1) Has three 16-bit ADCs for digitizing the gyroscope outputs and three 16-bit ADCs for digitizing the accelerometer outputs.
2) For precision tracking of both fast and slow motions,
    - It has user programmable gyroscope full scale range of ± 250, ± 500, ± 1000 and ± 2000 degree/second.
    - Has User programmable accelerometer full-scale range of ± 2g, ± 4g, ± 8g and ± 16g. (Where g = 9.81 m/s^2)
    - The VLOGIC reference pin is used to set the logic levels of its I^2C interface.
    - The I^2C address AD0 = 1101000 and AD1 = 1101001.

Initializing and configuring an MPU6050 gyro sensor via the I^2C Communication Protocol

```
//Setup the MPU-6050
if(type == 1){

  Wire.beginTransmission(address);          //Start communication with the gyro
  Wire.write(0x6B);                         //PWR_MGMT_1 register
  Wire.write(0x00);                         //Set to zero to turn on the gyro
  Wire.endTransmission();                   //End the transmission
```

The above code section initiates communication with the MPU6050 at the specified address “address”. Then it writes ‘0x00’ to the register ‘0x6B’, which is the Power management 1 register i.e., PWR_MGMT_1

Therefore, writing 0x00 to PWR_MGMT_1 register sets a bit to enable the gyro and start its operation.

```
Wire.beginTransmission(address);          //Start communication with the gyro
Wire.write(0x6B);                         //Start reading @ register 28h and auto increment with every read
Wire.endTransmission();                   //End the transmission
Wire.requestFrom(address, 1);             //Request 1 bytes from the gyro
while(Wire.available() < 1);              //Wait until the 1 byte is received
Serial.print(F("Register 0x6B is set to:"));
Serial.println(Wire.read(),BIN);
```

The above code initiates communication to start reading from the register 0x6B and sets up auto-increment with each read. Then it requests one byte from MPU6050 and waits until the byte is available. Once received, it prints the value of register 0x6B in binary Format (for user understanding).

```
Wire.beginTransmission(address);          //Start communication with the gyro
Wire.write(0x1B);                         //GYRO_CONFIG register
Wire.write(0x08);                         //Set the register bits as 00001000 (500dps full scale)
Wire.endTransmission();                   //End the transmission
```

Here it configures the gyroscope’s configuration register i.e., GYRO_CONFING; register -> 0x1B.

Then it writes 0x08 to configure the gyro for a full-scale range of ± 500

degrees/second.

(To know roles of each address, refer to the datasheet which is linked in the appendix)

Reading angular data (roll, pitch and yaw) from MPU6050 via I^2C protocol and compensating for calibration offsets when necessary

```
Wire.beginTransmission(address);                    //Start communication with the gyro
Wire.write(0x43);                                   //Start reading @ register 43h and auto increment with every read
Wire.endTransmission();                             //End the transmission
Wire.requestFrom(address,6);                        //Request 6 bytes from the gyro
while(Wire.available() < 6);                        //Wait until the 6 bytes are received
gyro_roll=Wire.read()<<8|Wire.read();               //Read high and low part of the angular data
if(cal_int == 2000)gyro_roll -= gyro_roll_cal;      //Only compensate after the calibration
gyro_pitch=Wire.read()<<8|Wire.read();              //Read high and low part of the angular data
if(cal_int == 2000)gyro_pitch -= gyro_pitch_cal;    //Only compensate after the calibration
gyro_yaw=Wire.read()<<8|Wire.read();                //Read high and low part of the angular data
if(cal_int == 2000)gyro_yaw -= gyro_yaw_cal;        //Only compensate after the calibration
```

The above code is used to start reading from the registers and enable auto increment mode so that subsequent reads automatically advance to next register.

Then it requests 6 bytes of data from MPU6050 and waits until all 6 bytes are available for reading. High and low parts of angular data for roll are read from the gyro sensor (This high byte s read first, left-shifted by 8 bits and then combined with the low byte using bitwise OR operation.

If calibration count (cal_int) equals 2000, it indicates that the calibration is completed and the offset gyro_roll_cal is subtracted from the raw gyro roll data. This is repeated for yaw and pitch angles.

We know that Arduino Uno R3 processor has 16MHz mode I^2C for communicating with all registers.

For communicating with MPU6050 the I^2C communication must be reduced to 400KHz of Arduino Uno R3. This is because MPU6050 has a I^2C of 400KHz.

We can control the clock frequency of I^2C bus by using TWBR register.

22.9 Register Description

22.9.1 TWBR – TWI Bit Rate Register

Bit	7	6	5	4	3	2	1	0	
(0xB8)	TWBR7	TWBR6	TWBR5	TWBR4	TWBR3	TWBR2	TWBR1	TWBR0	TWBR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bits 7...0 – TWI Bit Rate Register**

TWBR selects the division factor for the bit rate generator. The bit rate generator is a frequency divider which generates the SCL clock frequency in the Master modes. See "Bit Rate Generator Unit" on page 212 for calculating bit rates.

Bit Rate Generator Unit

This unit controls the period of SCL when operating in a Master mode. The SCL

period is controlled by settings in the TWI Bit Rate Register (TWBR) and the Prescaler bits in the TWI Status Register (TWSR). Slave operation does not depend on Bit Rate or Prescaler settings, but the CPU clock frequency in the Slave must be at least 16 times higher than the SCL frequency. Note that slaves may prolong the SCL low period, thereby reducing the average TWI bus clock period. The SCL frequency is generated according to the following equation:

$$\text{SCL frequency} = \frac{\text{CPU Clock frequency}}{16 + 2(\text{TWBR}) \cdot (PrescalerValue)}$$

- TWBR = Value of the TWI Bit Rate Register.
- PrescalerValue = Value of the prescaler

22.9.3 TWSR – TWI Status Register

Bit	7	6	5	4	3	2	1	0	
(0xB9)	TWS7	TWS6	TWS5	TWS4	TWS3	–	TWPS1	TWPS0	TWSR
Read/Write	R	R	R	R	R	R	R/W	R/W	
Initial Value	1	1	1	1	1	0	0	0	

- **Bits 1:0 – TWPS: TWI Prescaler Bits**

These bits can be read and written, and control the bit rate prescaler.

Table 22-7. TWI Bit Rate Prescaler

TWPS1	TWPS0	Prescaler Value
0	0	1
0	1	4
1	0	16
1	1	64

To calculate bit rates, see "Bit Rate Generator Unit" on page 212. The value of TWPS1...0 is used in the equation.

Therefore, for 400KHz with CPU clock frequency =16MHz and Prescaler value = 1

$$400000 = \frac{16000000}{16+(2)(TWBR).(1)}$$

400000(16 + 2 TWBR) = 16000000

16 + 2 TWBR = 40

$$\text{TWBR} = \frac{40-16}{2}$$

TWBR = 12

By setting TWBR =12 we can achieve 400KHz which can trigger I^2C communication in MPU6050. This is shown in the below code.

```
TWBR = 12;                      //Set the I2C clock speed to 400kHz.

#if F_CPU == 16000000L          //If the clock speed is 16MHz include the next code line when compiling
  clockspeed_ok = 1;            //Set clockspeed_ok to 1
#endif                          //End of if statement

if(TWBR == 12 && clockspeed_ok){
  Serial.println(F("I2C clock speed is correctly set to 400kHz."));
}
else{
  Serial.println(F("I2C clock speed is not set to 400kHz. (ERROR 8)"));
  error = 1;
}
```

To Check if gyro (MPU6050) is connected or not

Register 117 of the MPU6050 indicates "Who_am_I" which helps in identifying the MPU6050 to Arduino.

4.32 Register 117 – Who Am I
WHO_AM_I

Type: Read Only

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
75	117	-	WHO_AM_I[6:1]						-

Description:

This register is used to verify the identity of the device. The contents of *WHO_AM_I* are the upper 6 bits of the MPU-60X0's 7-bit I^2C address. The least significant bit of the MPU-60X0's I^2C address is determined by the value of the AD0 pin. The value of the AD0 pin is not reflected in this register.

The default value of the register is 0x68.

Bits 0 and 7 are reserved. (Hard coded to 0)

Parameters:

WHO_AM_I Contains the 6-bit I^2C address of the MPU-60X0.

The Power-On-Reset value of Bit6:Bit1 is 110 100.

```
//Search for the gyro and check the Who_am_I register
byte search_gyro(int gyro_address, int who_am_i){
  Wire.beginTransmission(gyro_address);
  Wire.write(who_am_i);
  Wire.endTransmission();
  Wire.requestFrom(gyro_address, 1);
  timer = millis() + 100;
  while(Wire.available() < 1 && timer > millis());
  lowByte = Wire.read();
  address = gyro_address;
  return lowByte;
}
```

Auto Levelling

Auto-Leveling :- (MPU-6050)

- From datasheet [MPU-6050 Product Specialization]

Page – 12 [Gyroscope specialization]

V_{DD} = 2.375V to 3.46V

V_{Logic} = 1.8V ±5% or V_{DD}

T_A = 25°C

→ Full Scale Range

- FS_SEL = 0 → ±250°/sec
- FS_SEL = 1 → ±500°/sec
- FS_SEL = 2 → ±1000°/sec
- FS_SEL = 3 → ±2000°/sec

→ Sensitivity Scale Factor

- FS_SEL = 0 → 131 LSB/(°/sec) — when angular motion of the gyro is 1°/sec
- FS_SEL = 1 → 65.5 LSB/(°/sec)
- FS_SEL = 2 → 32.8 LSB/(°/sec)
- FS_SEL = 3 → 16.4 LSB/(°/sec)

→ Gyroscope ADC Word Length → 16 bits

o/p of gyro is 16 bits signed value

→ [This is similar to Arduino integer. i.e, the variable / raw data stored needs to be declared as an integer]

- considering FS_SEL = 1 i.e ±500°/sec

Zaxis out = 65.5

Angular velocity = 1°/s

- If we rotate the gyro around its yaw axis and complete the full turn in exactly one-minute,

The o/p of the gyro at any moment will be Zaxis out = 393.

Because for every second the gyro uses 6°

∴ Zaxis out = 6°/s × 65.5

Zaxis out = 393 (raw gyro o/p)

[If we want to keep the o/p in °/sec we need to divide the raw gyro o/p by 65.5]

i.e., Angle in degrees = $\frac{\text{raw gyro o/p}}{65.5}$

- Now, when angular velocity = 500°/sec

Zaxis out = 500°/sec × 65.5

$\boxed{\text{Zaxis out} = 32750}$ (raw gyro o/p)

↳ This is almost max. value of an integer (Arduino) [−32,768 to +32,767]

- Before using the MPU6050 to quadcopter we need to calibrate the MPU6050 modules.

Calibration :-

- Initially the gyro gives default / Random values even though it is not moving.
- We need to make these values to zero, so that we get best results.
- We can achieve this, by taking average offset from multiple readings and subtract from the raw output.

Code:

```
for (int cal_int = 0; cal_int < 2000; cal_int ++)
{
    if (cal_int % 125 == 0)
    lcd.print(".");
    read_mpu_6050_data();
    gyro_x_cal += gyro_x;
    gyro_y_cal += gyro_y;
    gyro_z_cal += gyro_z;
    delay(3);
}
gyro_x_cal /= 2000;
gyro_y_cal /= 2000;
gyro_z_cal /= 2000;
```

→ taking 2000 reading for the gyro

- What happen when we add all the gyro output values every second?

Let us consider for 1min rotation;

At any moment in time (1min)

Zaxis out = 393 [angular velocity = 6°/sec]

For every sec in 1min;

393 × 60 = 23580

↳ $\frac{23580}{65.5}$ = 360°

The above calculation implies the total angle the gyro has travelled

→ For our quadcopter, we have considered 250Hz as our refresh rate.

i.e ESC's are corrected every 1/4 msec.

So, we can integrate the gyro's output every 4msec instead of every 1sec

i.e., Zaxis out / 250 / 65.5 to get travelled angle.

∴ Zaxis out / 250 / 65.5 = Zaxis out × 0.0000611

$\frac{1}{250} = \frac{0.004}{65.5}$

0.0000 61066

≈ 0.0000611

• In arduino code; we can write as,

angle_pitch += gyro_x * 0.0000611;

angle_roll += gyro_y * 0.0000611;

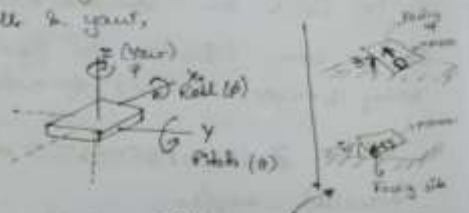
[Note: declare the quadcopter angle as float type]

- Considering for 1min rotation,
  - It takes 60 sec to complete the rotation
  - During this rotation, we have,

850 readings p/s × 60 sec = 15,000 readings

∴ 15000 × 393 × 0.0000611 = 360.1845°

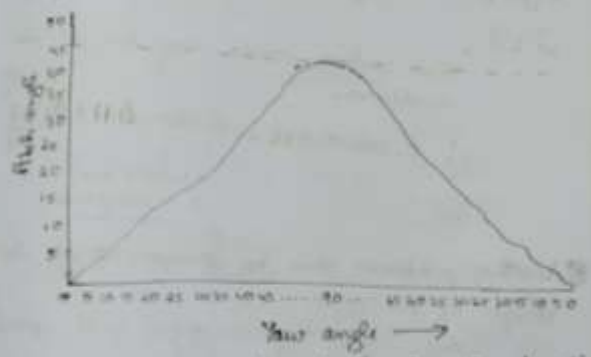
• Standard diagram we consider for pitch, roll & yaw,



⊛ One problem we might encounter is when the MPU6050 is inclined we get pitch and roll value, but when we rotate during the inclination the values of both pitch & roll remain same.

- This can be rectified if we couple the yaw data to pitch & roll.

For every 5° in yaw movement we can have the graph of pitch vs yaw as;



- The above graph is similar to sine function

∴ The function for transferring the pitch and roll angle is a sined function,

i.e

Angle roll = (Angle roll − Angle pitch × sin (gyro z × 0.00001066)

Angle pitch = (Angle pitch + Angle roll) × sin (gyro z × 0.00001066)

- The formula can be written in arduino as,

code;

angle_roll -= angle_pitch * sin (gyro_z * 0.00001066)

angle_pitch += angle_roll * sin (gyro_z * 0.00001066)

w.k.t;

Sine value needs an angle in radians.

$$\therefore 0.00001066 = \underbrace{0.0000611 \times \frac{3.142}{180}}_{= 1.0658 \times 10^{-6} \simeq 0.00001066}$$

⊛ Another problem can be encountered is the drift of the gyro.

Vibrations caused by the gyro will speed up the drift process.

PROJECT - Mass Flow Controller

Mass Flow Controller: Device used to measure and control the flow rate gas or liquid.

In general, it is a compact device that supplies one or more chemical compounds gases or liquid to a process by accurately controlling the flow rate.

Uses

1) Semiconductor Manufacturing: In fabrication process, MFC's are used to control the Flow rates of various gases such as dopants, etchants deposition precursors.
2) Chemical Vapor Deposition (CVD): It is a process which involves the deposition of thin films onto substrates by chemical reactions in the gas phase. MFC's are used to control the flow of precursor & Carrien gases. [for uniform film deposition etc.]
3) Gas Chromatography: It is a technique used for separating & analysing volatile compounds in a sample. MFC's are used to control the low rate of carrier gases such as helium or nitrogen, which carry sample through chromatographic column for separation & detection.
4) Biotechnology & pharmaceuticals: MFC's are used to control the flow rates of gases Backs as air, oxygen, nitrogen & Carbon dioxide.
5) Environmental monitoring: MFC's are used to control the flow rata of calibration gases for gas analysers & sensors This is to ensure accurate calibration & measure of air quality parameters such as concentrations & greenhouse gas emissions measure pollutant.
6) Research laboratories: MFC’s are used in material synthesis, gas-phase reactions & gas mixing. For precise control on experimental condition and investigating the effects of different gas flow chemical processes.
7) Flow Control Systems: Used in Heat treatment, surface coating & Chemical processing. MFC's play a major role in maintaining process Stability, optimizing efficiency & ensuring product quality.

Basic Principle:

MFC operates based on the principle of regulating the flow rate of a gas through a - controlling the mass How rate. System by controlling the mass flow rate.

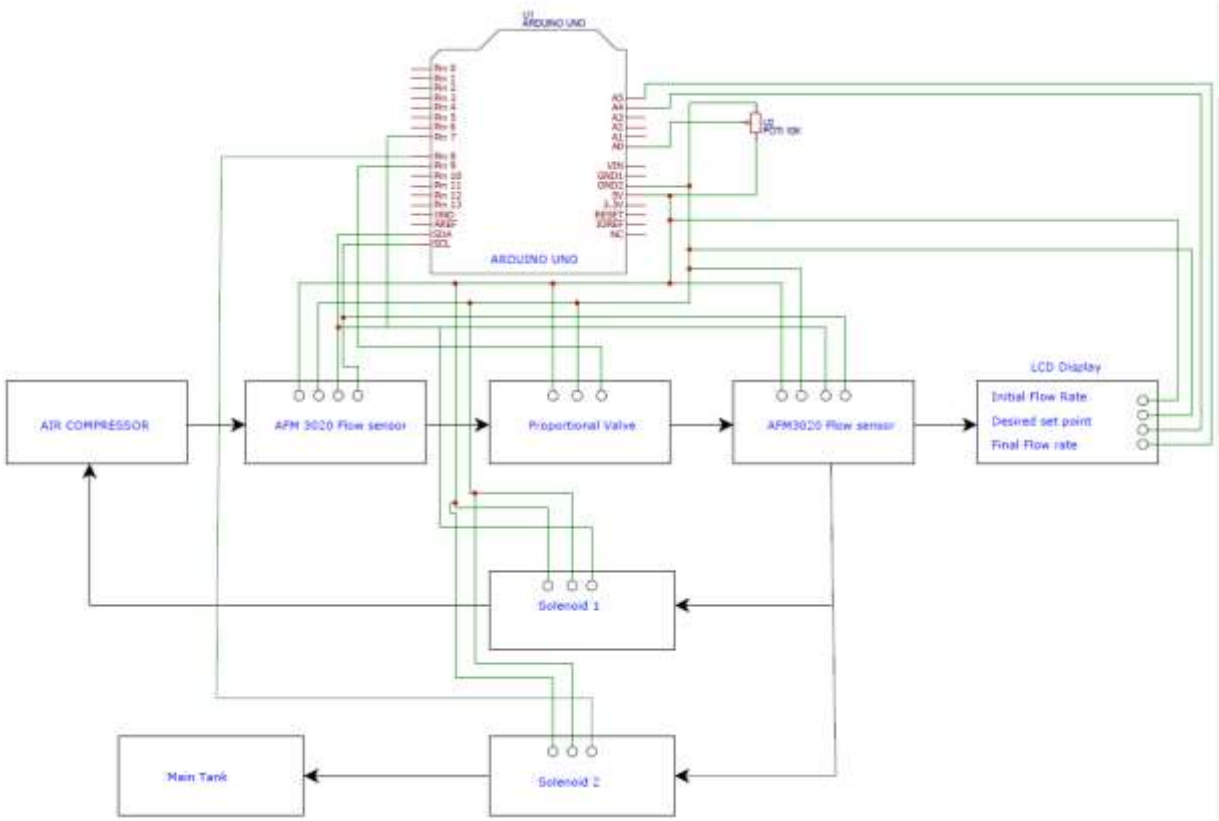
Types

- Thermal mass Flow Controllers.
- Pressure Based mass Flow Controllers.
- Differential Pressure-Based Mass Flow Controllers.
- Coriolis Mass Flow Controllers.
- Volumetric Flow Controllers.

Objectives:

1. Accurate Flow Measurement: Implementing a reliable system to accurately measure the mass flow rate of the gas using flow sensors, ensuring precise monitoring and control of the flow throughout the operation.
2. Proportional Valve Control: Developing an effective control mechanism that adjusts the proportional valve in real-time based on the desired flow rate, enabling smooth and responsive regulation of the gas flow.
3. User Interface and Feedback: Creating a user-friendly interface, such as an LCD display and potentiometer, to allow easy adjustment of the desired flow rate, while providing real-time feedback on the actual flow rate and system status.

Circuit Diagram:



Explanation

The system regulates air flow using an Arduino and various components to achieve a precise flow rate. Initially, air flows through a 1st AFM3020 flow sensor, which measures and reports the flow rate to the Arduino. The desired flow rate, set using a potentiometer, is then compared with the measured rate. The Arduino adjusts a proportional valve to achieve the desired flow. A 2nd AFM3020 flow sensor, placed downstream of the valve, verifies that the actual flow matches the setpoint. Two solenoid valves manage air flow: one blocks the air from reaching the main tank until the desired flow rate is achieved, while the other redirects the air back to the pump and closes when the setpoint is met. The 16x2 LCD display shows the flow rate from the 1st sensor, the desired setpoint, and the flow rate from the 2nd sensor.

Code Explanation:

MFC_AIR_FINAL.ino

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// Pin definitions
const int proportionalValvePin = 9;   // PWM pin for proportional valve control
const int solenoidValve1Pin = 7;      // Digital pin for solenoid valve 1
const int solenoidValve2Pin = 8;      // Digital pin for solenoid valve 2
const int potentiometerPin = A0;      // Analog pin for the potentiometer

// Flow sensor addresses (Assume both sensors are connected via I2C)
const int flowSensor1Address = 0x40;  // I2C address for AFM3020 Flow Sensor 1
const int flowSensor2Address = 0x41;  // I2C address for AFM3020 Flow Sensor 2

// Initialize the LCD
LiquidCrystal_I2C lcd(0x27, 16, 2);  // Adjust address to your LCD module
```

- Wire.h is used for I2C communication with the flow sensors.
- LiquidCrystal_I2C.h is for controlling the LCD via I2C.
- Pin numbers for various components are defined: PWM pin for the proportional valve, digital pins for solenoid valves, and an analog pin for the potentiometer.
- I2C addresses for the flow sensors are specified.
- The LCD is initialized with its I2C address (0x27), and it is set to a 16x2 display.

```
// Function prototypes
float readFlowSensor(int sensorAddress);
void adjustProportionalValve(float error);
void controlSolenoidValves(float flowRate2);
```

Prototypes for functions used later in the code are declared here. This tells the compiler that these functions will be defined later.

```
void setup() {
  Serial.begin(9600);

  // Initialize pins
  pinMode(proportionalValvePin, OUTPUT);
  pinMode(solenoidValve1Pin, OUTPUT);
  pinMode(solenoidValve2Pin, OUTPUT);

  // Initialize solenoid valves (Initially block the air to the main tank)
  digitalWrite(solenoidValve1Pin, LOW);  // Close valve 1 (block air to main tank)
  digitalWrite(solenoidValve2Pin, HIGH); // Open valve 2 (redirect air to pump)

  // Initialize I2C
  Wire.begin();

  // Initialize the LCD
  lcd.begin();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("Flow Control Sys");
}
```

- Serial.begin(9600); initializes serial communication for debugging.
- The pin modes for the proportional valve and solenoid valves are set to OUTPUT.
- Solenoid valves are initialized: Valve 1 is closed, and Valve 2 is opened.
- I2C communication is started with Wire.begin().
- The LCD is initialized and displays a welcome message.

```
void loop() {
  // Read the desired flow rate from the potentiometer
  int potValue = analogRead(potentiometerPin);
  float desiredFlowRate = map(potValue, 0, 1023, 0, 100);  // Adjust the range as necessary

  // Read flow rate from the first sensor
  float flowRate1 = readFlowSensor(flowSensor1Address);

  // Calculate the error between desired and actual flow rate
  float error = desiredFlowRate - flowRate1;

  // Adjust the proportional valve based on the error
  adjustProportionalValve(error);

  // Read flow rate from the second sensor
  float flowRate2 = readFlowSensor(flowSensor2Address);

  // Control the solenoid valves based on the flow rate from the second sensor
  controlSolenoidValves(flowRate2);

  // Display the values on the LCD
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("FS1: ");
  lcd.print(flowRate1, 1);
  lcd.print(" LPM");

  lcd.setCursor(0, 1);
  lcd.print("Set: ");
  lcd.print(desiredFlowRate, 1);
  lcd.print(" FS2: ");
  lcd.print(flowRate2, 1);

  // Short delay for stability
  delay(500);
}
```

- The potentiometer value is read and mapped to a desired flow rate (0 to 100 LPM in this case).
- The flow rate from the first sensor is read.
- The error between the desired and actual flow rates is calculated.
- The proportional valve is adjusted based on this error to regulate the flow rate.
- The flow rate from the second sensor is read to ensure the system's accuracy.
- Solenoid valves are controlled based on the second sensor's flow rate.
- The LCD is updated with the flow rates and desired setpoint.
- A short delay is added for stability.

```
float readFlowSensor(int sensorAddress) {
  Wire.beginTransmission(sensorAddress);
  Wire.write(0x00); // Command to read the flow rate (this depends on the AFM3020 datasheet)
  Wire.endTransmission();

  Wire.requestFrom(sensorAddress, 2);
  if (Wire.available() == 2) {
    uint16_t rawData = Wire.read() << 8 | Wire.read();
    float flowRate = (rawData / 32768.0) * 100.0;  // Convert to SLPM (depends on sensor calibration)
    return flowRate;
  } else {
    return -1.0;  // Return error if no data available
  }
}
```

Starts communication with the sensor using its I2C address. Sends a command to read the flow rate (you need to refer to the sensor datasheet for the correct command). Reads 2 bytes of data from the sensor and converts it to a flow rate value. Returns the flow rate, or -1.0 if there's an error.

```
void adjustProportionalValve(float error) {
  // Map error to PWM value (adjust as necessary for your valve)
  int pwmValue = constrain(map(error, -100, 100, 0, 255), 0, 255);
  analogWrite(proportionalValvePin, pwmValue);
}
```

Maps the error (difference between desired and actual flow rate) to a PWM value. The map function scales the error to a range suitable for the PWM output. The constrain function ensures the PWM value is within the valid range (0 to 255). The analogWrite function sets the PWM output to control the proportional valve.

```
void controlSolenoidValves(float flowRate2) {
  if (flowRate2 >= desiredFlowRate) {
    digitalWrite(solenoidValve1Pin, HIGH);  // Open valve 1 (allow air to main tank)
    digitalWrite(solenoidValve2Pin, LOW);   // Close valve 2 (stop redirecting air)
  } else {
    digitalWrite(solenoidValve1Pin, LOW);   // Close valve 1 (block air to main tank)
    digitalWrite(solenoidValve2Pin, HIGH);  // Open valve 2 (redirect air to pump)
  }
}
```

- Compares the flow rate from the second sensor with the desired flow rate.
- If the second sensor's flow rate is greater than or equal to the desired flow rate, it opens Valve 1 and closes Valve 2.
- If the second sensor’s flow rate is less, it closes Valve 1 and opens Valve 2.

CONCLUSION

Reflecting on my internship experience, I am deeply grateful for the transformative journey it has been. This opportunity has provided me with invaluable insights into the industry and has significantly contributed to my personal and professional development. Throughout this internship, I have had the chance to:

Acquire and Utilize Practical Skills: I was able to bring the theoretical knowledge from my academic studies to life in real-world scenarios. This hands-on experience has greatly enhanced my understanding of specific skills and concepts.

Collaborate Effectively: I learned to work harmoniously with diverse working styles and perspectives, which contributed to a more holistic and efficient work environment.

Overcome Challenges: Facing various challenges during my internship pushed me to develop strong problem-solving abilities and adaptability, allowing me to find innovative solutions.

Understand Company Culture: My time at CMTI provided me with a deep understanding of the company's values, mission, and culture. I saw firsthand how these elements drive success and create a positive work environment.

Beyond the technical skills, this internship has profoundly impacted my personal growth. I have become more self-confident, adaptable, and resilient qualities that will undoubtedly benefit me in my future endeavors.

Looking back, I can confidently say that this internship has been a rewarding experience that aligns perfectly with my long-term career aspirations. I am immensely thankful for the support and guidance from my mentors and colleagues, which made this journey so fulfilling.

As I move forward in my academic and professional journey, I will carry the valuable lessons and experiences from this internship with me. I am excited to leverage the knowledge and skills I have gained to make meaningful contributions to any future endeavors in the industry.

APPENDIX

Sl.No.	Datasheets and References	Link
1	ATMEL 8-bit Microcontroller with4/8/16/32K bytes in- System Programmable Flash	https://robu.in/wp-content/uploads/2018/11/Atmel-8271-8-bit-AVR-Microcontroller-ATmega48A-48P-1315288.pdf
2	8-bit AVR microcontroller with 32Kbytes in-System Programmable Flash	https://www.alldatasheet.com/datasheet-pdf/pdf/1132281/ATMEL/ATMEGA328P.html
3	Arduino Uno	https://docs.arduino.cc/hardware/uno-rev3
4	Nrf24l01 Transceiver	https://www.alldatasheet.com/datasheet-pdf/pdf/1489848/NORDIC/NRF24L01.html
5	MPU6050-Product Specification Revision 3.3 MPU6050 – Register Map and Descriptions Revision 4.2	https://www.alldatasheet.com/datasheet-pdf/pdf/517744/ETC1/MPU-6050.html https://www.alldatasheet.com/datasheet-pdf/pdf/1132807/TDK/MPU-6050.html
6	STM32F103x6 STM32F103x6 related datasheets	https://www.alldatasheet.com/datasheet-pdf/pdf/201585/STMICROELECTRONICS/STM32F103X6.html https://www.alldatasheet.com/view.jsp?Searchword=STM32F103X6
7	Flysky fs t6 user manual	https://robu.in/wp-content/uploads/2019/01/User-Manual.pdf

8	Flight Controller Code using Arduino Uno Videos	https://www.brokking.net/ymfc-al_downloads.html https://www.youtube.com/playlist?list=PL0K4VDicBzsibZqfa42DVxC8CGCMB7G2G
9	Flight Controller Code using STM32F103C8T6 Videos	https://www.brokking.net/ymfc-32_auto_downloads.html https://www.youtube.com/playlist?list=PL0K4VDicBzshwCpUHzIB6hOLQVkDFHbxC
10	Flight Controller Code using Arduino Uno Reference page	https://www.brokking.net/ymfc-al_main.html
11	Flight Controller Code using STM32F103C8T6 Reference page	https://www.brokking.net/ymfc-32_auto_main.html
12	GPS: u-blox 6 Receiver Description Including Protocol Specification	https://content.u-blox.com/sites/default/files/products/documents/u-blox6_ReceiverDescrProtSpec_%28GPS.G6-SW-10018%29_Public.pdf
13	Plant leaf disease detection video	https://drive.google.com/file/d/1TUzrBE7L5kbdQbNDdOh99PV3uQoSVxdz/view?usp=sharing